

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Тихоокеанский государственный университет»

М. С. Алёшин, В. В. Насыров

ВЫЧИСЛИТЕЛЬНАЯ ФИЗИКА

*Утверждено издательско-библиотечным советом университета
в качестве учебного пособия*

Хабаровск
Издательство ТОГУ
2016

УДК 531.1 (075.8)

ББК В3с51я7

А496

Р е ц е н з е н т ы:

проф. кафедры «Физика» Комсомольского-на-Амуре государственного технического университета, доктор физ.-мат. наук *Н. А. Хохлов*;

доц. кафедры «Высшая математика» Дальневосточного государственного университета путей сообщения, канд. физ.-мат. наук *А. И. Ливашвили*.

Алёшин, М. С.

А496 Вычислительная физика : учеб. пособие / М. С. Алёшин, В. В. Насыров. – Хабаровск : Изд-во Тихоокеан. гос. ун-та, 2015. – 80 с.
ISBN 978-5-7389-

В пособии рассмотрены алгоритмы обработки экспериментальных данных, способы расчёта траектории движения тела, карт векторных полей. Изложена краткая теория погрешностей, теоретические основы механического движения и колебательных процессов. Приведены основы численного интегрирования, моделирования векторных полей. Работа содержит теоретическую часть, программный код на языке программирования C++ с использованием библиотеки Qt, который можно использовать как основу для написания собственных программ, и набор заданий для самостоятельного решения.

Издание предназначено для выполнения лабораторных работ в компьютерном классе по курсу «Вычислительная физика», для студентов младших курсов, обучающихся по направлению «Физика». Может быть востребовано преподавателями курса «Вычислительная физика».

УДК 531.1 (075.8)

ББК В3с51я7

ISBN 978-5-7389-

© Тихоокеанский государственный университет, 2016

© Алёшин М. С., Насыров В. В., 2016

ВВЕДЕНИЕ

На данном этапе развития науки вычислительная техника играет огромную роль. Она является важной составной частью в процессе познания мира в целом и при изучении физики в частности. С одной стороны, выполнение современных экспериментов практически невозможно без использования компьютерной техники, которая применяется как для сбора и обработки экспериментальных данных, так и для их анализа. С другой стороны, развитие вычислительной техники привело к возникновению нового метода исследования — вычислительного (компьютерного) эксперимента, который стал новым инструментом научного познания. Для проведения вычислительного эксперимента необходимо наличие численных моделей различных физических процессов и явлений.

В пособии рассмотрены некоторые возможности табличного процессора *Microsoft Excel*, которые могут успешно использоваться в учебном процессе студентами-физиками для обработки массивов данных. Также большое внимание уделено способам решения некоторых классических задач механики, молекулярной физики, электричества и магнетизма методами вычислительной физики. Приведены вычислительные алгоритмы с примерами их реализации на языке программирования *C++* в среде разработки *Qt Creator*.

1. КОМПЬЮТЕРНАЯ ОБРАБОТКА ЭКСПЕРИМЕНТАЛЬНЫХ ДАННЫХ

§ 1. Погрешности измерений

Введение

Выполнение любого физического эксперимента подразумевает измерение различных физических величин с последующей обработкой полученных результатов. Никакие измерения не могут быть абсолютно точными, поэтому при любом измерении измеренное значение величины всегда отличается от ее истинного значения. Задачей экспериментатора является не только нахождение самой величины, но и оценка допущенной при измерении погрешности. В зависимости от свойств и причин возникновения различают следующие виды погрешностей:

- систематические,
- случайные,
- промахи (грубые ошибки).

Систематическими называются погрешности, которые при многократных измерениях, проводящихся одним и тем же методом с помощью одних и тех же измерительных приборов, остаются постоянными. Систематические погрешности вызываются факторами, действующими одинаковым образом, они соответствуют отклонению измеренного значения от истинного всегда в одну сторону.

Систематические погрешности могут быть обусловлены, во-первых, неисправностью или неправильной работой на используемых приборах (например, неправильной установкой “нуля”). Во-вторых, их причиной может быть несовершенство используемой методики измерения или неучёт постоянных факторов, влияющих на исследуемое явление. Например, можно получать завышенные значения температуры плавления кристалла, если проводить измерения при повышенном внешнем давлении.

Помимо погрешностей, возникающих в процессе измерений, систематическими являются погрешности, связанные с применением приближенных формул, и ошибки, обусловленные отличием реального объекта от принятой модели. Как правило, факторы, вносящие систематические погрешности, могут быть изучены до начала измерений, а результат измерения может быть уточнен за счет внесения соответствующих поправок, которые дают возможность исключать влияние систематических погрешностей.

Случайными называются погрешности, которые при многократных измерениях в одинаковых условиях изменяются непредсказуемым образом.

Случайные ошибки обусловлены множеством неконтролируемых причин, действие которых неодинаково в каждом опыте. В результате этого при измерении одной и той же величины несколько раз подряд в одинаковых условиях получается целый ряд значений этой величины, отличающихся от истинного значения случайным образом.

Случайные ошибки всегда присутствуют в эксперименте.

Природа случайных погрешностей может быть различной: флуктуации нулевого положения указателя измерительного прибора; несовершенство органов чувств экспериментатора (например, невозможность включить секундомер точно в нужный момент); случайные неконтролируемые изменения внешних воздействий – температуры, влажности, давления; наводки в электрической цепи и т. д., которые практически невозможно учесть.

Прوماхи (или грубые погрешности) проявляются обычно в резком отклонении результата отдельного измерения от остальных. Промахи обусловлены главным образом недостаточным вниманием экспериментатора или неисправностями средств измерения. Результаты таких измерений отбрасываются.

Кроме того, источниками экспериментальных погрешностей выступают приборы, используемые при измерении величины. Поэтому отдельно выделяют инструментальную (или приборную) погрешность – разность между показаниями прибора и истинным значением измеряемой величины. Инструментальная погрешность может содержать случайную и систематическую составляющие.

Расчет случайной погрешности в прямых измерениях

Кратко рассмотрим теорию случайных ошибок. В ее основе лежит три положения: 1) ошибки измерения принимают непрерывный ряд значений; 2) при большом количестве измерений одной и той же величины ошибки разного знака встречаются одинаково часто; 3) большие по абсолютной величине ошибки встречаются реже, чем малые, то есть вероятность появления ошибки уменьшается с ростом ее величины.

Пусть некоторая величина X измеряется N раз:

$$X_1, X_2, \dots, X_N.$$

Задача обработки экспериментальных данных результатов измерений заключается в определении границы интервала, в котором заключено истинное значение измеряемой величины. Принята следующая форма записи результата измерений какой-либо величины:

$$X = X_{\text{cp}} \pm \Delta X. \quad (1.1)$$

За наиболее вероятное значение величины X принимают среднее арифметическое значение результатов измерений:

$$X_{\text{cp}} = \frac{1}{N} \sum_{i=1}^N X_i. \quad (1.2)$$

Чем больше число измерений, тем ближе среднее значение к истинному.

При наличии в измерениях случайной погрешности абсолютная погрешность ΔX по методу Стьюдента определяется следующим образом:

$$\Delta X = t_{\beta, N} \left(\frac{\sigma_N}{\sqrt{N}} \right); \quad \sigma_N = \sqrt{\frac{\sum_{i=1}^N (X_{\text{cp}} - X_i)^2}{(N - 1)}}, \quad (1.3)$$

где σ_N – стандартное отклонение; $t_{\beta, N}$ – коэффициент Стьюдента; β – надежность, величина, равная вероятности, с которой доверительный интервал включает в себя истинное значение величины X .

В лабораториях физического практикума принято значение надежности $\beta = 0,95$.

Коэффициент Стьюдента $t_{\beta,N}$ является поправочным и применяется для корректировки доверительного интервала при небольшом числе измерений ($N < 30$). Коэффициент Стьюдента $t_{\beta,N}$ может быть рассчитан в рамках теории вероятности (табл. 1.1).

Таблица 1.1. Значение коэффициента Стьюдента $t_{\beta,N}$ при надёжности $\beta = 0,95$ для различного числа измерений N

N	2	3	4	5	6	7	8	9	10	15	20	100	∞
$t_{0,95,N}$	12,7	4,3	3,2	2,8	2,6	2,4	2,4	2,3	2,3	2,1	2,1	2,0	2,0

Для примера обозначим на числовой оси точками результаты измерения некоторой величины X . Они группируются вокруг среднего значения X_{cp} (рис. 1.1).

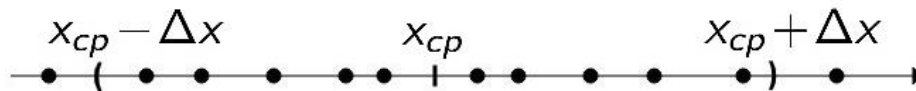


Рис. 1.1. Обозначение доверительного интервала на числовой оси

Круглыми скобками обозначим доверительный интервал, внутри которого находятся 19 возможных экспериментальных значений из 20, тогда надёжность $\beta \approx \frac{19}{20} = 0,95$.

Отметим, что приведенный метод расчета случайной погрешности **всегда носит оценочный характер**.

§ 2. Графическое представление результатов измерений

Основные правила оформления графиков

В экспериментальной физике для визуального представления измеренных результатов применяют графики, которые дают возможность наглядного восприятия разного рода функциональных зависимостей в больших массивах данных.

Часто графики строят для определения наклона прямых, отображающих зависимости между двумя наборами переменных. Этот метод используют для оценки результатов, полученных более точными методами.

При построении графиков через имеющиеся экспериментальные данные можно провести как теоретическую, так и экспериментальную кривые.

При оформлении графиков необходимо выполнять следующие правила.

1. График должен содержать надпись, из которой было бы ясно физическое содержание представленной закономерности.
2. Масштабы и начала отсчета по координатным осям выбираются так, чтобы график изображения зависимости занимал большую часть поля чертежа. При этом на пересечении осей не обязательно должны находиться нулевые значения величин (рис. 2.1).

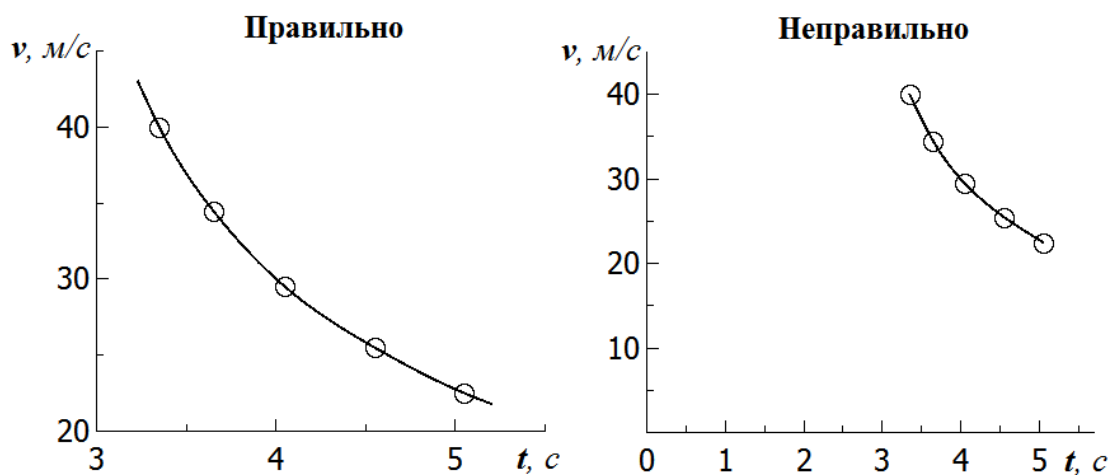


Рис. 2.1. Примеры выбора точки начала отсчёта

3. На осях координат откладываются равноотстоящие друг от друга деления масштаба так, чтобы было удобно работать с графиком. Значения, полученные в эксперименте, не указываются (рис. 2.2).

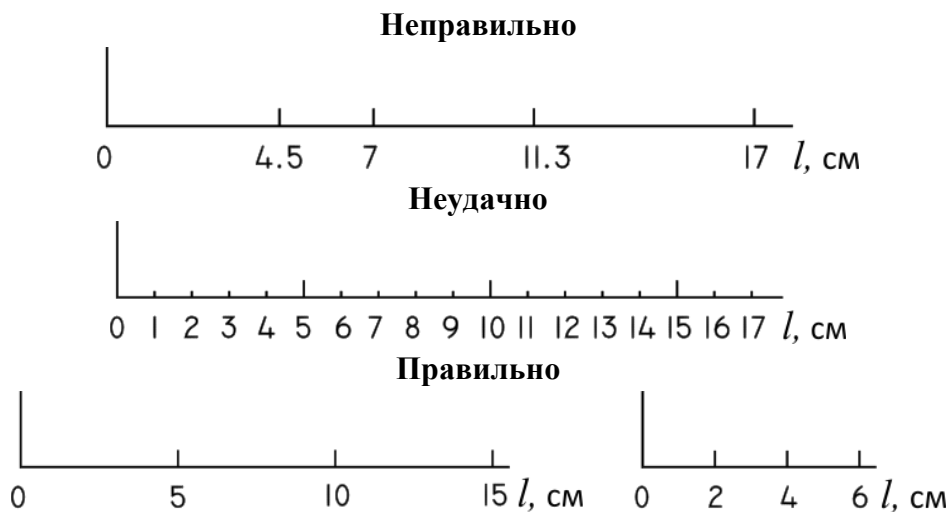


Рис. 2.2. Примеры выбора интервала делений на числовой оси

4. В конце координатных осей обязательно указываются условные обозначения откладываемых величин и, через запятую, их единицы измерения.
5. Экспериментальная кривая проводится плавно через доверительные интервалы всех или большинства экспериментальных точек так, чтобы экспериментальные точки наиболее близко и равномерно располагались с разных сторон кривой (рис. 2.3).

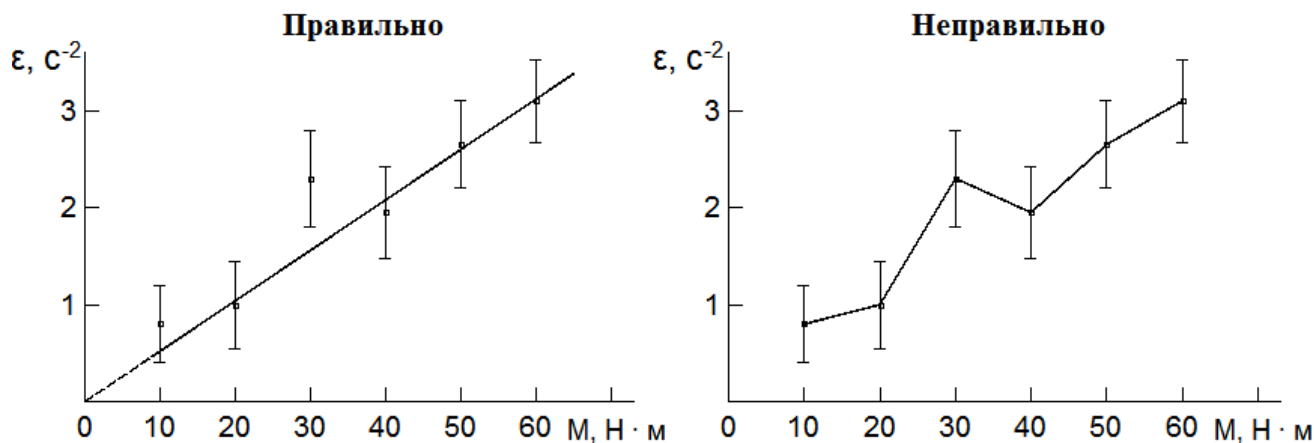


Рис. 2.3. Примеры построения экспериментальной кривой

Метод наименьших квадратов

Метод наименьших квадратов (МНК) представляет наиболее распространенный способ построения кривых, наилучшим образом соответствующих экспериментальным данным.

Предположим, что в опыте получена совокупность экспериментальных данных $\{X_i\}$ и $\{Y_i\}$ и теоретическая модель предполагает их линейную зависимость. Параметры a и b подбираются так, чтобы уменьшить отклонения экспериментальных точек (X_i, Y_i) от графика функции. В МНК требуется найти такую функцию $y(x) = ax + b$, чтобы сумма квадратов отклонений этой функции от опытных данных была минимальной:

$$\Phi(a, b) = \sum_{i=1}^N ((aX_i + b) - Y_i)^2 \Rightarrow \min. \quad (2.1)$$

Получаем классическую задачу на экстремум функции двух неизвестных a и b . Необходимое условие экстремума

$$\begin{cases} \Phi'_a = 0 \\ \Phi'_b = 0. \end{cases} \quad (2.2)$$

Продифференцировав функцию $\Phi(a, b)$ по a и по b , получим следующую систему линейных уравнений:

$$\begin{cases} a \sum_{i=1}^N X_i^2 + b \sum_{i=1}^N X_i = \sum_{i=1}^N X_i Y_i \\ a \sum_{i=1}^N X_i + Nb = \sum_{i=1}^N Y_i \end{cases} \quad (2.3)$$

Разделим каждое уравнение на число наблюдений N . Получим

$$\begin{cases} a \frac{\sum_{i=1}^N X_i^2}{N} + b \frac{\sum_{i=1}^N X_i}{N} = \frac{\sum_{i=1}^N X_i Y_i}{N} \\ a \frac{\sum_{i=1}^N X_i}{N} + b = \frac{\sum_{i=1}^N Y_i}{N} \end{cases} \quad (2.4)$$

Определим средние величины:

$$\begin{aligned} X_{\text{cp}} &= \frac{1}{N} \sum_{i=1}^N X_i, & Y_{\text{cp}} &= \frac{1}{N} \sum_{i=1}^N Y_i, \\ (X^2)_{\text{cp}} &= \frac{1}{N} \sum_{i=1}^N X_i^2, & (XY)_{\text{cp}} &= \frac{1}{N} \sum_{i=1}^N X_i Y_i. \end{aligned} \quad (2.5)$$

Тогда система примет вид

$$\begin{cases} a(X^2)_{\text{cp}} + bX_{\text{cp}} = (XY)_{\text{cp}} \\ aX_{\text{cp}} + b = Y_{\text{cp}} \end{cases} \quad (2.6)$$

Решая систему, нетрудно получить параметры a и b в виде

$$a = \frac{(XY)_{\text{cp}} - X_{\text{cp}}Y_{\text{cp}}}{(X^2)_{\text{cp}} - (X_{\text{cp}})^2}, \quad b = Y_{\text{cp}} - aX_{\text{cp}}. \quad (2.7)$$

В итоге получим функцию $y = ax + b$, которая является линией наилучшего приближения для данной совокупности точек $\{X_i, Y_i\}$.

Расчет случайной погрешности средствами Excel

Табличный процессор MS Excel позволяет автоматизировать процесс расчета экспериментальных погрешностей. С использованием встроенных функций Excel расчет доверительного интервала проводится следующим образом:

1. Рассчитывается среднее значение

=СРЗНАЧ(число1; число2; ...)

число1, число2, ... — аргументы, для которых вычисляется среднее.

2. Рассчитывается стандартное отклонение

=СТАНДОТКЛОНП(число1; число2; ...)

число1, число2, ... — аргументы, для которых ищем стандартное отклонение.

3. Рассчитывается абсолютная погрешность

=ДОВЕРИТ(альфа; станд_откл; размер)

альфа — уровень значимости, используемый для вычисления уровня надежности, $\alpha = 1 - \beta$, т. е. $\alpha = 0,05$ означает, что коэффициент надежности $\beta = 0,95$;

станд_откл — стандартное отклонение, предполагается известным;

размер — размер выборки.

Пусть проведен опыт по измерению сопротивления металлического проводника. В опыте посредством однократных измерений измерена зависимость силы тока, проходящего через проводник, как функции напряжения на концах этого проводника и получены следующие данные:

<i>U, В</i>	4,737	10,876	15,661	20,878	24,574	30,171	36,254	40,684	44,834	45,526
<i>I, мА</i>	89	229	292	419	473	562	773	759	925	942

Сопротивление проводника можно вычислить по закону Ома $R = U/I$, получая следующие значения:

<i>R, Ом</i>	53,5	47,5	53,6	49,8	51,9	53,7	46,9	53,6	48,5	48,4
--------------	------	------	------	------	------	------	------	------	------	------

Будем рассматривать данный набор значений $\{R_i\}$ как серию экспериментальных данных и применим метод Стьюдента для ее обработки.

Вводим исходные данные в Excel. Вычисляем сопротивление и обрабатываем данные по формулам, приведенным в табл. 2.1.

Таблица 2.1. Пример расчёта доверительного интервала в редакторе *Microsoft Excel*

Пример расчёта					Используемые формулы
	A	B	C	D	D
1	N	U, В	I, А	R, Ом	R, Ом
2	1	4.737	0.089	53.225	=B2/C2
3	2	10.876	0.229	47.493	=B3/C3
4	3	15.661	0.292	53.634	=B4/C4
5	4	20.878	0.419	49.828	=B5/C5
6	5	24.574	0.473	51.953	=B6/C6
7	6	30.171	0.562	53.685	=B7/C7
8	7	36.254	0.773	46.900	=B8/C8
9	8	40.684	0.759	53.602	=B9/C9
10	9	44.834	0.925	48.469	=B10/C10
11	10	45.526	0.942	48.329	=B11/C11
12	Xcp			50.712	=CP3НАЧ(D2:D11)
13	σ			2.788	=СТАНДОТКЛОН(D2:D11)
14	ΔX			1.728	=ДОВЕРИТ(0.05;D13;10)
15	ε, %			4.945	=CPОТКЛ(D2:D11)/D12*100

С учетом правил округления в рассматриваемом примере для сопротивления получен доверительный интервал $R = 50,71 \pm 1.73$ Ом.

Изучим полученную экспериментальную зависимость $I(U)$. Для построения графика будем использовать мастер диаграмм.

На первом шаге (рис. 2.4) нужно выделить столбцы значений, по которым будет строиться график. Затем во вкладке *Вставка* из блока *Диаграммы* вызвать диалоговое окно *Вставка диаграммы* (кнопка в правом нижнем углу блока), в этом окне выбрать тип диаграммы – *Точечная*.

На втором шаге, используя вкладки *Конструктора* работы с диаграммами, добавляем элементы на график: указываем величины, отложенные по осям, название диаграммы. При необходимости во вкладке *Формат* можно изменить текущие элементы диаграммы.

В редакторе *Microsoft Excel* реализована возможность проведения аппроксимирующих кривых по методу наименьших квадратов при помощи *линий тренда*. Для добавления линии тренда следует нажать правой кнопкой мыши по одной из точек построенного графика и в контекстном меню выбрать пункт *Добавить линию тренда* (рис. 2.5).

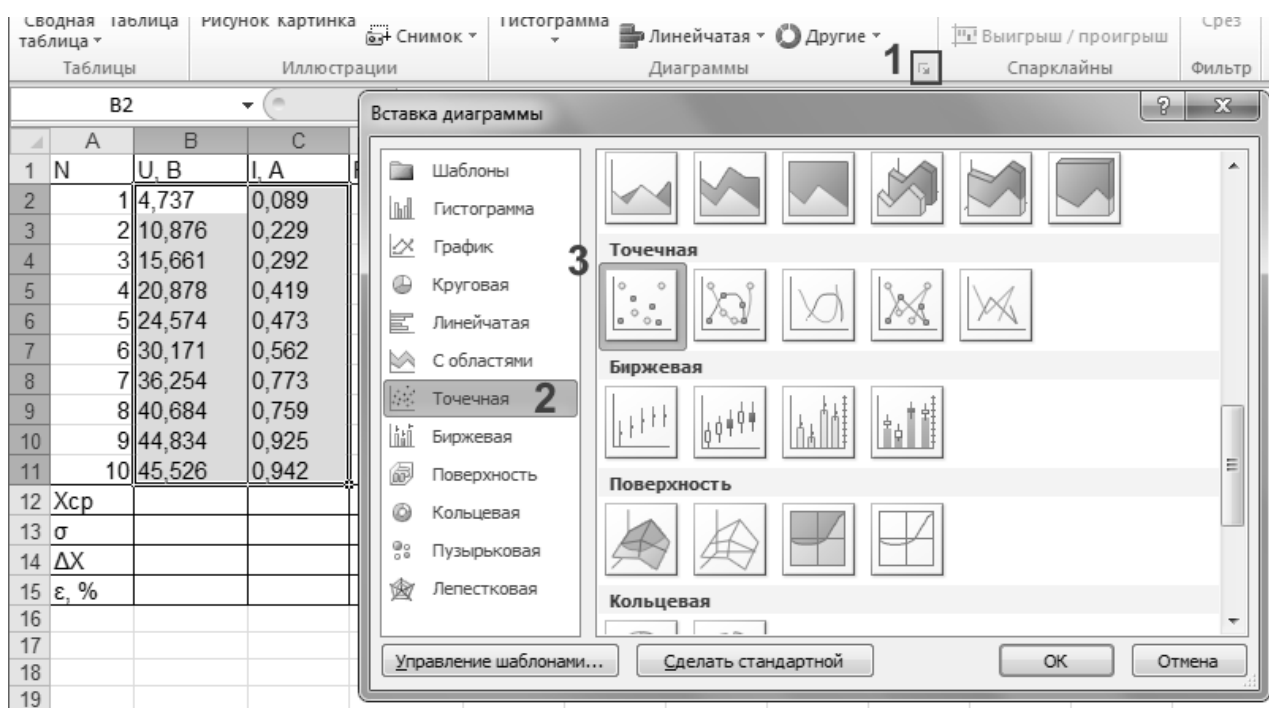


Рис. 2.4. Добавление графика

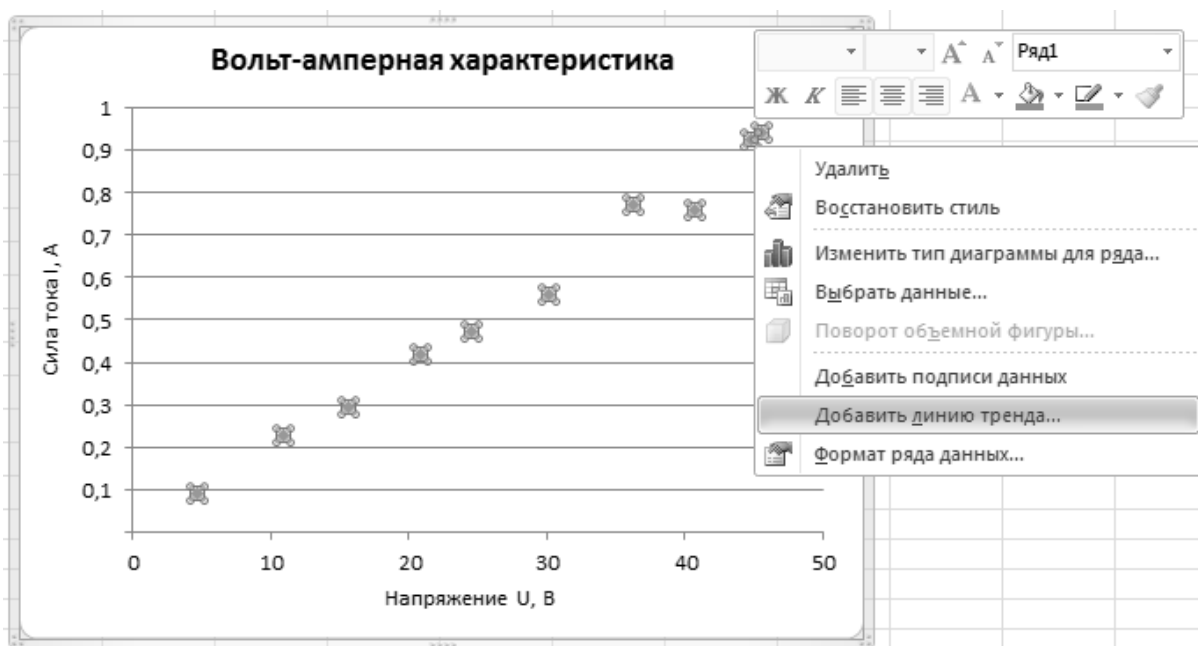


Рис. 2.5. Вызов контекстного меню для добавления линии тренда к имеющемуся графику

После этого в появившемся диалоговом окне будет предложено произвести форматирование добавляемой аппроксимирующей линии. Результат добавления линии, построенной по методу наименьших квадратов, приведён на рис. 2.6.

Для корректного отображения следует задать некоторые параметры добавленной линии (рис. 2.7). В частности, чтобы линия строилась по методу наименьших квадратов, необходимо выбрать в качестве типа *Линейная*.

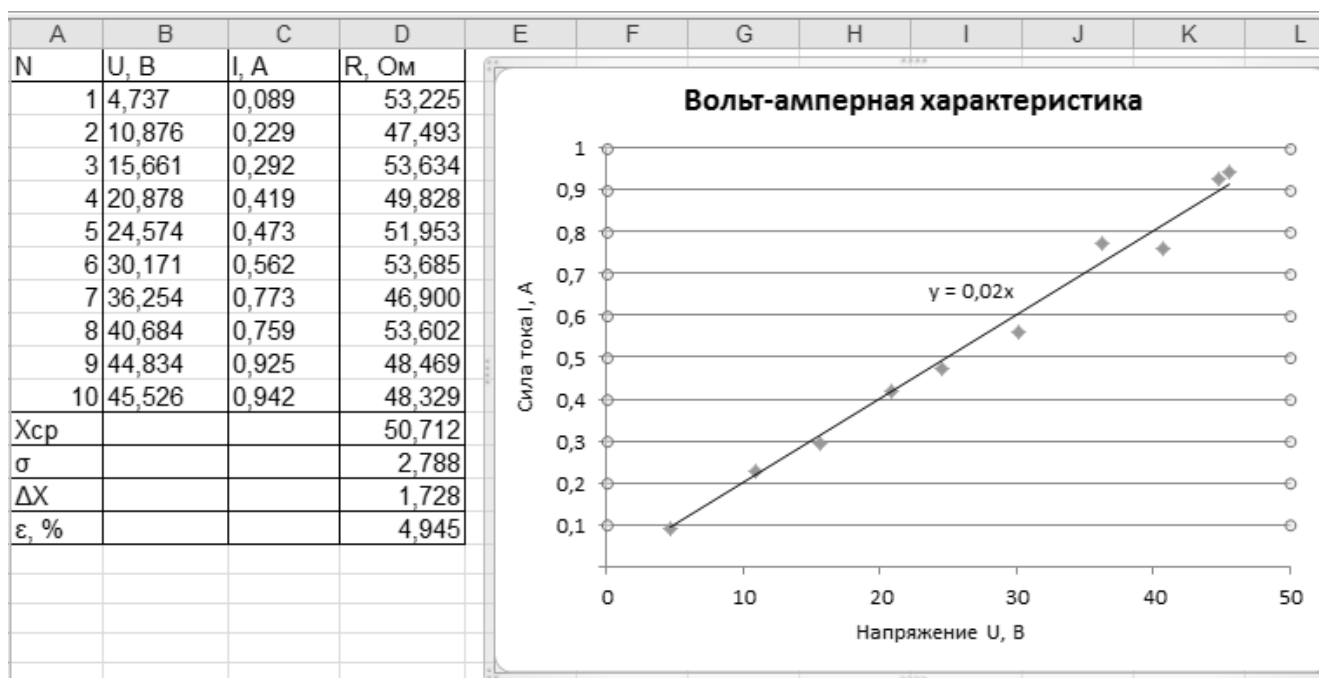


Рис. 2.6. Результат добавления линии тренда

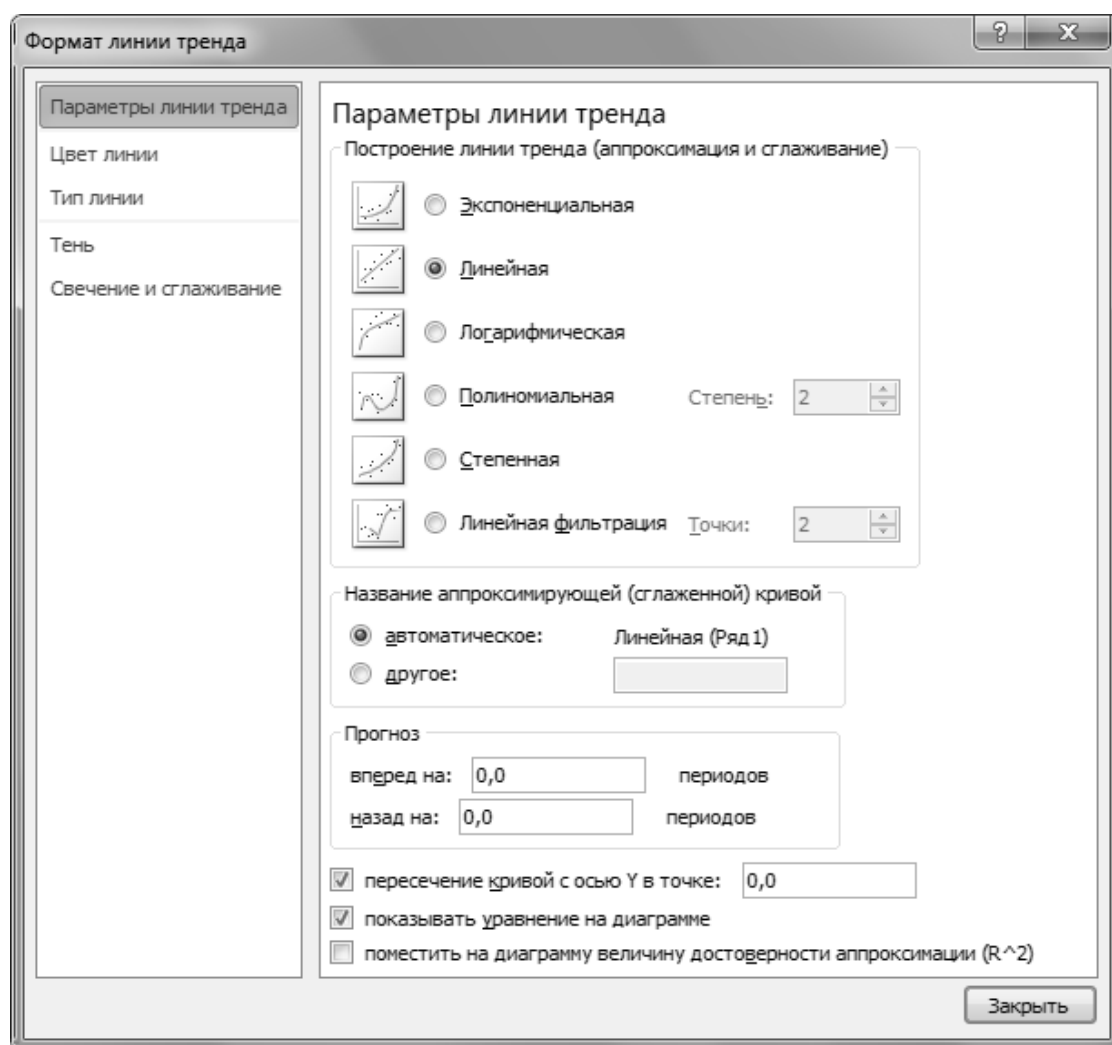


Рис. 2.7. Окно форматирования линии тренда

Применяя функции форматирования диаграмм, получим график, соответствующий необходимым требованиям оформления. Отметим, что существует возможность автоматического расчета параметров a и b по формуле (2.7), и вывода зависимости в числовом виде.

На графике (рис. 2.8) приведена линейная зависимость $y(x) = 0.02x$, из которой следует, что средняя проводимость проводника $G = 0,02 \text{ Ом}^{-1}$. Среднее значение сопротивления проводника, полученное из графика $R = 1/0,02 = 50 \text{ Ом}$, лежит в границах рассчитанного доверительного интервала.

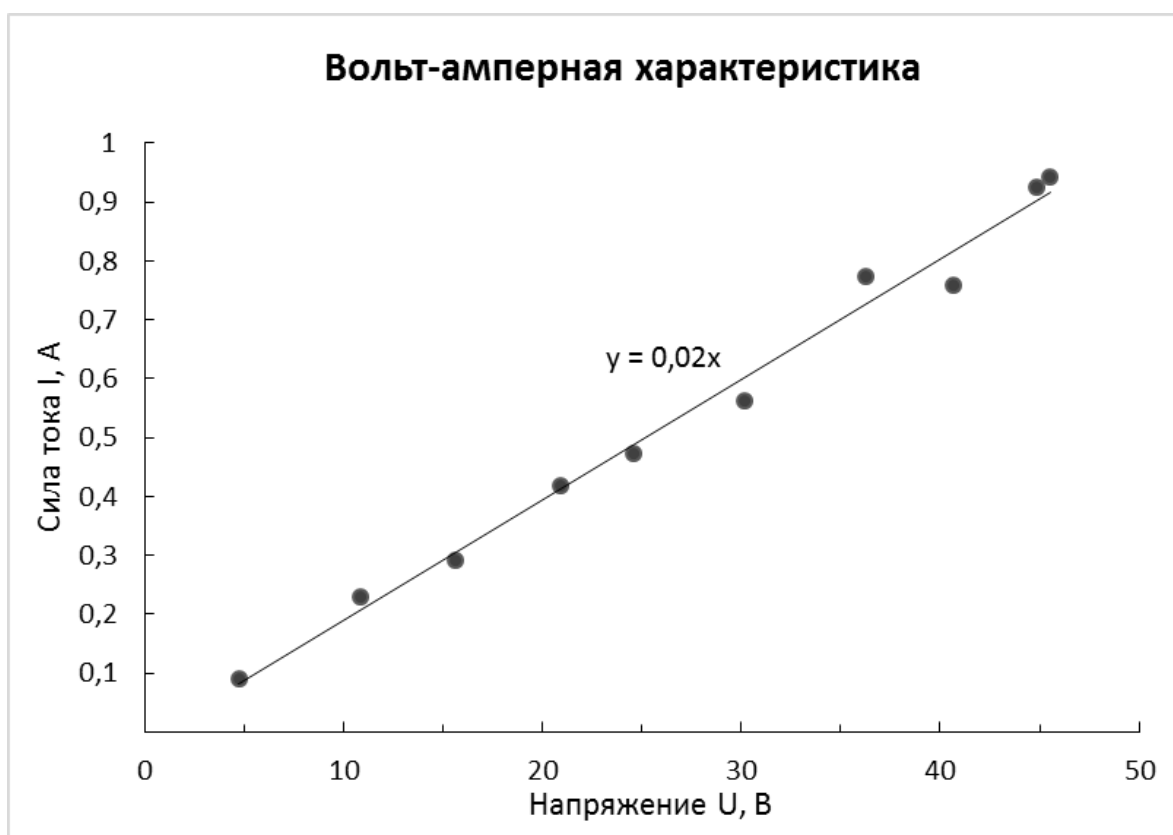


Рис. 2.8. Окончательный вид графика

Расчет погрешности при косвенных измерениях

При измерении величины косвенным методом предполагается, что известна математическая модель

$$Y = F(X_1, X_2, \dots, X_N),$$

связывающая искомую величину Y с величинами X_i , измеряемыми непосредственно. Далее предполагается, что выполнена обработка всех прямых измерений, т. е. определены доверительные интервалы для величин X_i :

$$X_i = \bar{X}_{\text{ср}i} \pm \Delta X_i. \quad (2.1)$$

Погрешность величины y определяется по формуле

$$\Delta Y = \sqrt{\sum_{i=1}^N \left(\Delta X_i \frac{\partial Y}{\partial X_i} \right)^2}, \quad (2.2)$$

где $Y_{\text{ср}} = F(X_{\text{ср}1}, X_{\text{ср}2}, \dots, X_{\text{ср}N})$.

Задания для самостоятельного выполнения

Лабораторная работа 1

Тема: Обработка прямых измерений в Excel

Задание:

1. Обработать заданный набор экспериментальных данных методом Стьюдента.
2. Построить экспериментальные кривые методом наименьших квадратов.
3. Определить значение искомой величины из графика, сравнить с рассчитанным средним.

Вариант 1

В опыте с помощью точного динамометра определяют силу F , при которой бруски различной массы m , сделанные из одинакового материала, лежащие на горизонтальной шероховатой поверхности, приходят в движение. Посредством однократных измерений получена зависимость:

F , мН	165,60	207,95	234,41	265,11	286,42	338,04	365,42	400,97	441,19	479,61
m , г	50,0	60,0	70,0	80,0	90,0	100,0	110,0	120,0	130,0	140,0

По этим данным строят зависимость силы F от массы m , из второго закона Ньютона $F = F_{\text{тр}} = \mu mg$ определяют коэффициент трения μ .

Вариант 2

В опыте к пружине, длина которой $l = 10$ см, подвешивают грузики различной массы и измеряют величину растяжения пружины Δl . Посредством однократных измерений получена зависимость:

$\Delta l, \text{мм}$	15,9	20,4	22,6	25,8	29,8	34,8	40,4	48,0	56,6	67,3
$m, \text{г}$	50,0	60,0	70,0	80,0	90,0	100,0	125,0	150,0	175,0	200,0

По этим данным строят зависимость растяжения Δl от силы $F_T = mg$, возникающей под действием грузика, из закона Гука и второго закона Ньютона $F_{\text{упр}} = k\Delta l = mg$ определяют жёсткость пружины k .

Вариант 3

В опыте определяют время t , за которое пружинный маятник совершает 10 колебаний, при подвешивании грузиков разной массы m . Посредством однократных измерений получена зависимость:

$t, \text{с}$	8,90	10,07	11,13	11,54	12,31	13,62	13,79	15,00	16,82	17,80
$m, \text{г}$	100,0	125,0	150,0	175,0	200,0	225,0	250,0	300,0	350,0	400,0

По этим данным строят зависимость квадрата периода колебаний T^2 от массы грузика m , из формулы для периода колебаний пружинного маятника $T = 2\pi\sqrt{\frac{m}{k}}$ определяют жёсткость пружины k .

Вариант 4

В опыте определяют время t , за которое математический маятник совершает 10 колебаний, при изменении длины нити l , на которой подвешен грузик. Посредством однократных измерений получена зависимость:

$t, \text{с}$	10,72	12,10	12,40	13,57	14,33	15,45	16,88	17,76	19,21	20,12
$l, \text{см}$	30,0	35,0	40,0	45,0	50,0	60,0	70,0	80,0	90,0	100,0

По этим данным строят зависимость квадрата периода колебаний T^2 от длины нити l , из формулы для периода колебаний математического маятника $T = 2\pi\sqrt{\frac{l}{g}}$ определяют ускорение свободного падения g .

2. КОМПЬЮТЕРНОЕ МОДЕЛИРОВАНИЕ ФИЗИЧЕСКИХ ЯВЛЕНИЙ

§ 3. Вычисление определенных интегралов

Методы численного интегрирования

Для вычисления определенных интегралов можно использовать различные численные методы, например метод прямоугольников, трапеций, Симпсона.

Метод средних прямоугольников. В методе область интегрирования разбивается на отрезки с некоторым шагом Δ и площадь под графиком функции на каждом -ом отрезке считается равной площади соответствующего прямоугольника

$$S_i = f\left(x_i + \frac{\Delta}{2}\right) \Delta, \quad (3.1)$$

где $\Delta = x_{i+1} - x_i$ (рис. 3.1).

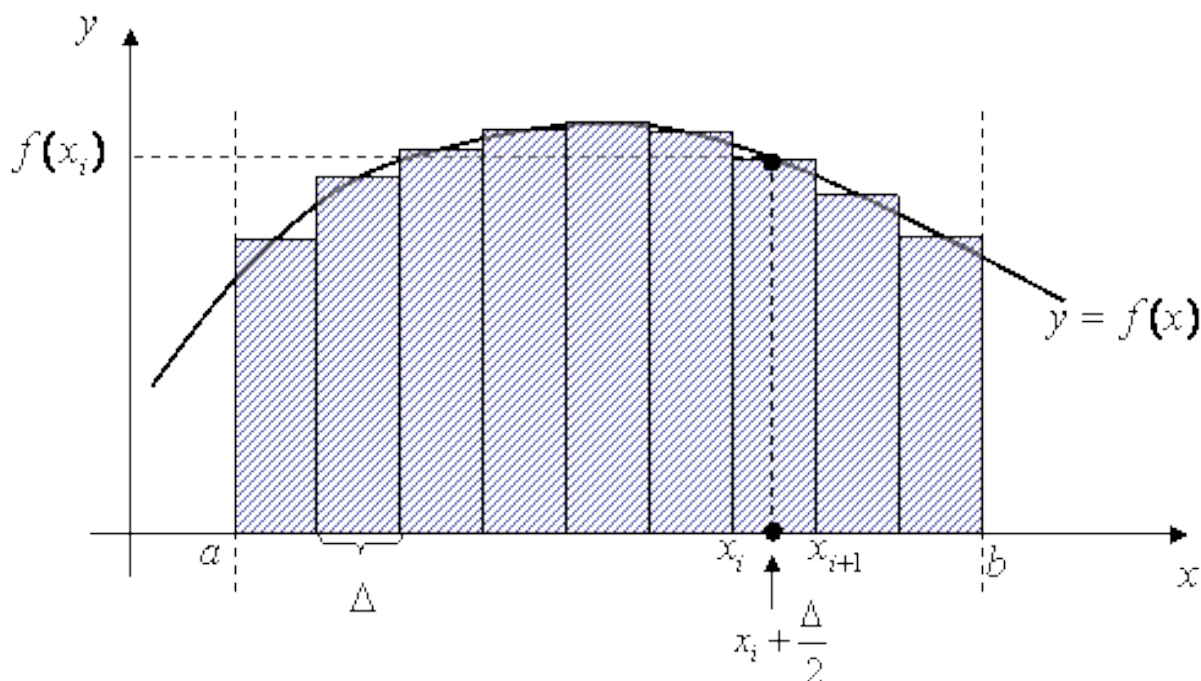


Рис. 3.1. Иллюстрация метода прямоугольников: значение интеграла на отрезке $[a, b]$ равно сумме площадей прямоугольников шириной Δ

Можно показать, что при использовании метода прямоугольников расчетная формула принимает следующий вид:

$$I_N = \int_a^b f(x) dx \approx h \sum_{i=0}^{N-1} f\left(a + h \cdot \left(i + \frac{1}{2}\right)\right), \quad (3.2)$$

где $h = (b - a)/N = \Delta$ – шаг разбиения; N – количество точек разбиения.

Для контроля точности количество точек разбиения удваивается, повторно выполняется расчет интеграла. Дробление исходного интервала прекращается, когда достигнута требуемая точность:

$$\varepsilon < \frac{|I_{2N} - I_N|}{3}. \quad (3.3)$$

При выполнении работы можно использовать процедуры, приведенные в приложении.

Метод трапеций. Метод заключается в замене подынтегральной функции на элементарном отрезке на многочлен первой степени на каждом элементарном отрезке. Другими словами, производится замена подынтегральной функции линейной, и площадь под графиком функции аппроксимируется прямоугольными трапециями.

Таким образом при использовании метода трапеций расчетная формула в случае равномерной сетки принимает следующий вид:

$$\begin{aligned} I_N &= \int_a^b f(x) dx \approx \sum_{i=0}^{N-1} \frac{f(a + h \cdot i) + f(a + h \cdot (i + 1))}{2} h = \\ &= \frac{h}{2} \sum_{i=0}^{N-1} [f(a + h \cdot i) + f(a + h \cdot (i + 1))]. \end{aligned} \quad (3.4)$$

где $h = (b - a)/N = \Delta$ – шаг разбиения; N – количество точек разбиения.

Можно далее упростить выражение, заметив, что в слагаемых с номерами i и $i+1$ содержатся одинаковые значения $f(a + h(i + 1))$. Тогда получим формулу Котеса:

$$\int_a^b f(x) dx \approx h \cdot \left(\frac{f(a) + f(a + Nh)}{2} + \sum_{i=0}^{N-1} f(a + i \cdot h) \right). \quad (3.5)$$

Как и в любом методе численного интегрирования для контроля точности количество точек разбиения может увеличиваться, пока не будет достигнута требуемая точность. В случае, если количество узлов сетки удваивается на следующем шаге вычислений, то рационально использование формулы

$$I_{2N} = \frac{I_N}{2} + \frac{h}{2} \sum_{i=0}^{N-1} f\left(a + \left(i + \frac{1}{2}\right)h\right) \quad (3.6)$$

для уменьшения времени, затрачиваемого на вычисления.

Метод Симпсона. Суть метода заключается в замене подынтегральной функции на элементарном отрезке интерполяционным многочленом второй степени. Иными словами, площадь под графиком функции на отрезке аппроксимируется параболой.

Интеграл на элементарном отрезке $h = b - a$ через формулу Симпсона будет равен

$$\int_a^b f(x) dx \approx \frac{h}{6} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right), \quad (3.7)$$

где $f(a)$, $f\left(\frac{a+b}{2}\right)$ и $f(b)$ – значения функции на концах отрезка и в его середине.

Для вычисления интеграла на интервале $[a, b]$ его разбивают на чётное число отрезков $N = 2n$ одинаковой длины и применяют формулу Симпсона на каждом из них. В результате получается выражение для исходного интеграла:

$$I_N = \int_a^b f(x) dx \approx \frac{h}{3} \left(f(a) + f(b) + 2 \sum_{i=1}^{n-1} f(a + 2ih) + 4 \sum_{i=1}^n f(a + (2i - 1)h) \right), \quad (3.8)$$

где $h = \frac{b-a}{N}$ – величина шага.

Также формулу для интегрирования можно записать в другом виде:

$$\int_a^b f(x) dx \approx \frac{h}{3} \sum_{i=1,2}^{N-1} [f(a + (i - 1)h) + f(a + ih) + f(a + (i + 1)h)], \quad (3.9)$$

где запись $i = 1, 2$ означает, что индекс меняется с шагом, равным двум, начиная с единицы.

В листинге 1 приведён код программы, вычисляющей интеграл функции на интервале методом средних прямоугольников.

Листинг 1. Расчёт определённого интеграла методом прямоугольников

```
/******  
#include <math.h>  
  
double F(double x) //Подынтегральная функция  
{  
    return 1 /* Объявление функции */;  
}  
  
// процедура вычисления интеграла;  
// a, b – пределы интегрирования  
// n – число узлов сетки  
double CalcIntegral(double a, double b, int n)  
{  
    int i;  
    double result, h;  
    result = 0;  
    h = (b - a) / n; //Шаг сетки  
  
    for(i = 0; i < n; i++)  
    {  
        //Вычисляем функцию в средней точке и добавляем в сумму  
        result += F(a + h * (i + 0.5));  
    }  
  
    result *= h;  
  
    return result;  
}  
/******
```

Решение задач на распределение Максвелла.

Функция распределения Максвелла

$$f(u) = \frac{4}{\sqrt{\pi}} \exp(-u^2) u^2 \quad (3.10)$$

позволяет определить вероятность молекуле идеального газа иметь скорость v в некотором интервале скоростей, где $u = v/v_B$ – относительная скорость, $v_B = \sqrt{2kT/m}$ – наиболее вероятная скорость, $k = 1,38 \cdot 10^{-23}$ Дж/К – постоянная Больцмана, T – температура газа, m – масса частиц (молекул).

При этом доля молекул идеального газа с относительными скоростями в интервале от u до $u+du$ определяется интегралом:

$$\frac{\Delta N}{N} = \int_u^{u+\Delta u} f(u) du = \frac{4}{\sqrt{\pi}} \int_u^{u+\Delta u} \exp(-u^2) u^2 du. \quad (3.11)$$

Так как молекула всегда будет обладать некоторой скоростью, то количество молекул со скоростями в интервале $[0, \infty]$ будет равно полному числу молекул, т. е.

$$\frac{\Delta N[0, \infty]}{N} = 1. \quad (3.12)$$

Поэтому должно выполняться условие нормировки функции Максвелла:

$$\int_0^{\infty} f(u) du = 1. \quad (3.13)$$

Для нахождения среднего значения относительной скорости молекул необходимо просуммировать значения скоростей u_i молекул с весом $\frac{\Delta N_i}{N}$, определяющим долю молекул, обладающих этими скоростями

$$\langle u \rangle = \sum_{i=1} u_i \frac{\Delta N_i}{N}. \quad (3.14)$$

В случае непрерывного распределения по скоростям молекул получаем

$$\langle u \rangle = \int_0^{\infty} u \frac{dN}{N} = \int_0^{\infty} u \cdot f(u) du. \quad (3.15)$$

Задания для самостоятельного выполнения

Лабораторная работа 2

Тема: Распределение Максвелла

Задание:

1. Вычислить определенные интегралы от функции Максвелла при помощи формул трапеций, Симпсона с заданной точностью.
2. Построить график функции распределения. На графике обозначить область интегрирования.
3. Проверить условие нормировки функции распределения (3.12).
4. В разработанной программе реализовать решение задачи на определение искомой величины (см. табл. 3.1).

Указание. Пример создания в среде *Qt Creator* простого приложения для работы с графиками см. в прил. I.

Таблица 3.1. Варианты для выполнения лабораторной работы 2

Вар.	Метод*	Задание**
1	Т	Какая часть молекул <i>водорода</i> при заданной температуре t обладает скоростями от v_1 до v_2 ($t = 0^\circ \text{C}$, $v_1 = 2000 \text{ м/с}$, $v_2 = 2100 \text{ м/с}$)
2	С	
3	Т	Какая часть молекул <i>водорода</i> при заданной температуре T имеет кинетическую энергию выше W_0 ($T = 1000 \text{ К}$, $W_0 = 3 \cdot 10^{-20} \text{ Дж}$)
4	С	
5	Т	Какая часть молекул <i>кислорода</i> при заданной температуре t обладает скоростями от v_1 до v_2 ($t = 0^\circ \text{C}$, $v_1 = 100 \text{ м/с}$, $v_2 = 110 \text{ м/с}$)
6	С	
7	Т	Какая часть молекул <i>кислорода</i> при заданной температуре T имеет кинетическую энергию выше W_0 ($T = 1600 \text{ К}$, $W_0 = 6,65 \cdot 10^{-20} \text{ Дж}$)
8	С	
9	Т	Какая часть молекул <i>азота</i> при заданной температуре t обладает скоростями от v_1 до v_2 ($t = 150^\circ \text{C}$, $v_1 = 300 \text{ м/с}$, $v_2 = 325 \text{ м/с}$)
10	С	

* Т – метод трапеций, С – метод Симпсона.

** Величины, для которых в задании указано значение в скобках, должны быть доступны для изменения пользователем в диалоговом окне приложения.

§ 4. Математическое моделирование движения и расчёт траектории

Введение

Одной из классических задач для моделирования физических процессов является расчет траектории движущейся точки в некоторых силовых полях. Силы могут зависеть от координат точки, времени и скорости точки. Отметим, что к исследованию траекторий движения отдельных частиц или макроскопических тел часто сводятся задачи моделирования достаточно сложных физических явлений.

Обычно для вычисления траектории решается система уравнений общего вида:

$$\begin{cases} \frac{d\vec{r}}{dt} = \vec{V}, \\ \frac{d\vec{V}}{dt} = \frac{\vec{F}}{m}. \end{cases} \quad (4.1)$$

где $\vec{r} = (x, y, z)$ и $\vec{V} = (V_x, V_y, V_z)$ – радиус-вектор и скорость частицы массы m ; $\vec{F} = (F_x, F_y, F_z)$ – действующая на неё сила. Очевидно, что первое уравнение в системе – это просто определение скорости как перемещения в единицу времени, а второе представляет собой основной закон динамики (II закон Ньютона) для нерелятивистского движения. Предполагается, что сила $\vec{F}$ известна в каждой точке пространства в любой момент времени.

Для однозначного решения системы дифференциальных уравнений требуется задать соответствующие начальные условия:

$$\begin{cases} \vec{r} = \vec{r}_0 \\ \vec{V} = \vec{V}_0 \end{cases} \quad (4.2)$$

т. е. положение тела в момент времени $t_0 = 0$, принятый за начало отсчёта, и его скорость в этот момент.

Если вместо двух систем векторных уравнений (4.1), (4.2) использовать их проекции на координатные оси, то в случае трёхмерной задачи получим шесть дифференциальных уравнений и шесть уравнений начальных условий (в проекци-

ях на декартовы оси). Итого шесть пар уравнений. Каждую такую пару уравнений можно представить обобщённо в виде

$$\begin{cases} \frac{dY}{dt} = S(Y, t) \\ Y = Y_0, \text{ при } t = t_0 \end{cases}. \quad (4.3)$$

Метод Эйлера

Алгоритм численного решения таких уравнений на промежутке $[t_0, t_0 + T]$ сводится к вычислению значений непрерывной функции $Y(t)$ лишь в дискретные моменты времени $t_k = k\Delta t$, где Δt – временной шаг, k – целое положительное число (номер шага). Значение функции Y в момент времени t_{k+1} вычисляется по ее значению в предыдущий момент времени t_k с помощью разложения в ряд Тейлора вблизи точки $Y(t_k)$. Ограничимся первым приближением данного разложения, на котором основан *метод Эйлера*. Этот метод является простейшим вычислительным алгоритмом, суть которого сводится к замене точной формулы (4.3) приближенным соотношением

$$Y(t_{k+1}) = Y(t_k) + S(Y(t_k), t_k) \Delta t. \quad (4.4)$$

Полученное по этой формуле значение функции $Y(t_{k+1})$ отличается от точного на величину ΔY . Геометрически же численная аппроксимация представляет собой замену истинной зависимости $Y(t)$ ломаной, которая совпадает с истинной функцией только в начальной точке t_0 . Расхождение истинной кривой и ломаной линии увеличивается с каждым шагом (рис. 4.1).

Проверка точности вычислительной схемы может быть осуществлена несколькими способами:

1. *Сравнение полученного численного решения с известным аналитическим решением* (при некоторых, обычно предельных, значениях параметров задачи).
2. *Сравнение результатов, полученных при вычислениях с меньшим шагом*. Вид траектории не должен меняться при достижении достаточной точности вычислений.

3. *Проверка выполнения законов сохранения.* Такой способ применим в случае, когда в задаче есть интегралы движения (например, энергия).

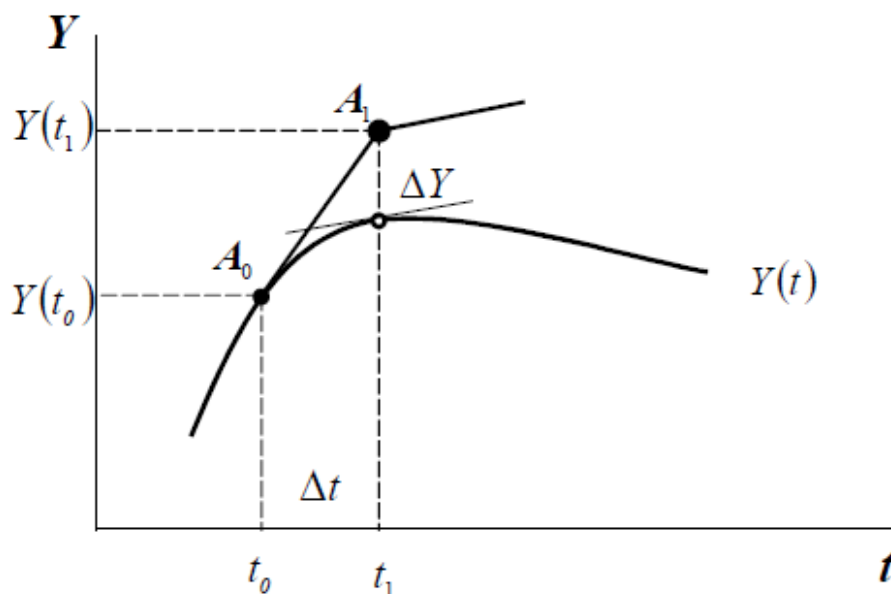


Рис. 4.1. Геометрическое объяснение возникновения погрешности вычислений по методу Эйлера

Движение физических тел в поле сил

Рассмотрим задачу движения тела на модели двумерного движения материальной точки в вертикальной плоскости в однородном поле тяготения. Предполагая, что на материальную точку действует сила тяжести $\vec{P} = \{0, -mg\}$, составим систему дифференциальных уравнений (4.1):

$$\begin{cases} \frac{dV_x}{dt} = 0, \\ \frac{dV_y}{dt} = -g, \\ \frac{dx}{dt} = V_x, \\ \frac{dy}{dt} = V_y. \end{cases} \quad (4.5)$$

Данная система имеет точное аналитическое решение:

$$y = y_0 + V_{y0}t - \frac{gt^2}{2},$$

$$x = x_0 + V_{x0}t,$$

$$V_y = V_{y0} - gt,$$

$$V_x = V_{x0},$$

где x_0, y_0 – начальные координаты и V_{y0}, V_{x0} начальные скорости точки, определяют начальные условия системы (4.5).

Для приближенного численного решения этой системы можно использовать разные численные методы. Рассмотрим метод конечных разностей, в котором бесконечно малые приращения переменных заменяют малыми конечными приращениями. Иными словами воспользуемся *методом Эйлера*:

$$\begin{cases} \frac{V_x(t + \Delta t) - V_x(t)}{\Delta t} = 0, \\ \frac{V_y(t + \Delta t) - V_y(t)}{\Delta t} = -g, \\ \frac{x(t + \Delta t) - x(t)}{\Delta t} = V_x(t), \\ \frac{y(t + \Delta t) - y(t)}{\Delta t} = V_y(t). \end{cases} \quad (4.6)$$

После несложных преобразований, получим систему алгебраических уравнений:

$$\begin{cases} V_x(t + \Delta t) = V_x(t), \\ V_y(t + \Delta t) = V_y(t) - g\Delta t \\ x(t + \Delta t) = x(t) + V_x(t)\Delta t, \\ y(t + \Delta t) = y(t) + V_y(t)\Delta t. \end{cases} \quad (4.7)$$

Отметим, что для решения полученных уравнений необходимо задание начальных условий, а именно: координат $x(t_0)$ и $y(t_0)$ материальной точки в начальный момент времени, значения её начальной скорости $V(t_0)$ и направления движения, или, что то же самое, проекций начальной скорости на соответствующие оси

$$V_x(t_0) = V(t_0)\cos(\alpha_0),$$

$$V_y(t_0) = V(t_0)\sin(\alpha_0),$$

где α_0 – угол между вектором $\vec{V}(t_0)$ и осью Ox .

Алгоритм расчета траектории движения тела:

1. *Задаётся начальное положение тела – координаты (x_0, y_0) .*
2. *Определяются проекции ускорения на оси координат (исходя из сил, действующих на тело в данной точке). В общем случае ускорение тела зависит от многих факторов и задается как функция от времени, скорости и координат точки.*
3. *Путём решения уравнений движения методом Эйлера с заданным шагом dt по времени определяются:*
 - 3.1. *Значения проекций скорости тела в данной точке*
 - 3.2. *Координаты тела в следующей точке x_{i+1}, y_{i+1} .*

Выбор величины dt определяется необходимой точностью расчета, возможностями вычислительной техники и может уточняться при решении задачи. Шаги 1-3 повторяются по циклу, при этом на каждой итерации определяется новое положение тела.

Примечание.

Для уменьшения погрешностей расчетной схемы скорость и ускорения на участке можно интерполировать их средними значениями, т. е. после определения ускорения в новой точке вместо выражения

$$v_{x_i} = v_{x_{i-1}} + a_{x_i} dt \quad (4.8)$$

использовать

$$v_{x_i} = v_{x_{i-1}} + \frac{(a_{x_i} + a_{x_{i-1}})}{2} dt, \quad (4.9)$$

аналогично для новой координаты использовать выражение

$$x_i = x_{i-1} + \frac{(v_{x_i} + v_{x_{i-1}})}{2} dt. \quad (4.10)$$

Приведём примерный код программы. Для удобства работы с векторными величинами введём структуру *vector*, которая будет хранить два вещественных параметра: горизонтальную и вертикальную составляющие вектора (мы ограничимся двухмерным случаем).

Листинг 2. Объявление структуры *vector*

```
struct vector {  
    double x; // горизонтальная составляющая  
    double y; // вертикальная составляющая  
};
```

Таким образом, мы сможем хранить координаты тела и проекции его скорости в переменных типа *vector*.

Для изображения анимации движения на форме приложения необходимо также определить процедуры рисования физического тела и его стирания (для того, чтобы отображать только текущее положения тела и удалять предыдущее). Назовём эти процедуры, *Draw* и *Wipe* соответственно. Их параметрами будут координаты тела. Реализация этих процедур зависит в первую очередь от среды разработки приложения, а также от классов и компонент, используемых в конкретной среде. Поэтому здесь мы не приводим код данных процедур.

Присвоение начальных значений можно поместить в отдельную процедуру, например, *Initialize*. Определим процедуру *Step* расчёта координат тела на отдельном временном шаге, которая будет вызываться в цикле на каждом шаге в процедуре *Move*. Тогда примерный код программы будет реализовываться листингом 3.

Листинг 3. Программная реализация движения тела в поле сил тяжести.

```
/* **** */  
#include <math.h> // для работы с математическими функциями  
  
double dt = 10; // временной шаг в миллисекундах  
double v0; // начальная скорость  
double angle; // начальное направление скорости  
double g = 9.81; // ускорение свободного падения  
vector v; // вектор скорости  
vector r; // радиус-вектор  
bool stop; // флаг окончания расчётов  
  
// процедура инициализации начальных параметров  
void Initialize()  
{  
    // определение проекций ускорения  
    a.x = 0; a.y = -g;  
}
```

```

// начальное положение тела
r.x = 0;    r.y = 0;

// модуль начальной скорости
v0 = 10; // например
// угол наклона начальной скорости к горизонтальной оси
angle = 45/180*M_PI; // например

// определение проекций начальной скорости
v.x = v0*sin(angle);    v.y = v0*cos(angle);

// отображение тела в начальной точке
Draw(r);
}

// процедура расчёта координат
void Step()
{
    // определение проекций ускорения
    a.x = 0;    a.y = -g;
    // определение проекций скорости по значениям ускорения
    v.x += a.x*dt;    v.y += a.y*dt;
    // определение новых координат тела
    r.x += v.x*dt;    r.y += v.y*dt;
}

// анимация движения
void Move()
{
    // инициализация начальных параметров
    initialize();

    // непосредственно анимация
    while (!stop) {
        // стираем изображение физического тела "из прошлого"
        Wipe(r);
        // расчёт новых координат
        Step();
        // отображение тела в текущей точке
        Draw(r);
        // задержка времени до следующей прорисовки
        sleep(dt);
    }
}

/*****/

```

Примечания:

- В данном листинге расчёт траектории реализован в цикле *while*, выполнение которого прекращается, когда логическая переменная *stop* принимает значение *true*. Изменение значения данного флага можно осуществить через нажатие кнопки на форме.
- Для правильного отображения анимации во времени использована функция задержки выполнения *sleep(dt)*, однако в зависимости от среды разработки данная задержка может реализовываться иными функциями. В разных программных пакетах они могут иметь разное название и принадлежать разным классам.
- Возможна реализация анимации иными средствами, например, использованием класса таймера, который присутствует практически во всех средах программирования. Пример создания модели равномерного движения тела с помощью библиотеки *Qt* приведён в прил. II.

Рассмотрим некоторые модельные задачи на расчёт траектории физического тела, движущегося в поле некоторых сил.

Движение тела в поле сил тяжести с учётом сил трения

Помимо силы тяжести $\vec{P}$ на движущееся тело в воздухе действует также сила сопротивления $\vec{F}_c$. Она всегда направлена против скорости движения (рис. 4.2), её величина пропорциональна характерной площади S , плотности среды ρ и квадрату скорости V :

$$F_c = C_x \frac{\rho V^2}{2} S, \quad (4.11)$$

где C_x – безразмерный аэродинамический коэффициент сопротивления, получается из критериев подобия, например, чисел Рейнольдса и Фруда в аэродинамике. Определение характерной площади зависит от формы тела, и в простейшем случае, которому соответствует сферическое тело (шар), её принимают равной площади поперечного сечения.

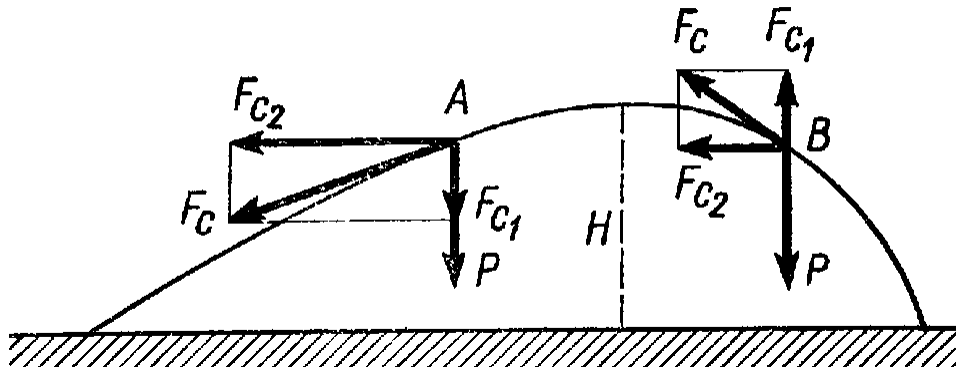


Рис. 4.2. Траектория движения тела в поле сил тяжести с учётом сопротивления вязкой среды. Обозначены направления сил, действующих на тело в точках траектории А и В

Таким образом, в условиях движения сферического тела в среде с постоянной плотностью величина

$$k = C_x \frac{\rho}{2} S \quad (4.12)$$

всюду одинакова. В задаче моделирования эту величину можно подобрать эмпирическим путём. Тогда

$$F_c = kV^2, \quad (4.13)$$

где k – коэффициент сопротивления тела о воздух.

Тогда в расчёте проекций ускорения необходимо помимо силы тяжести учесть проекции силы сопротивления на оси.

Движение заряда в электрическом поле неподвижного центра

В случае движения заряженной частицы с зарядом e и скоростью V в электрическом поле (рис. 4.3) действующая на неё сила равна $\vec{F} = e\vec{E}$.

Если источником поля $\vec{E}$ является точечный или сферический заряд Q , помещенный в начало координат, то сила задаётся формулой

$$\vec{F} = \frac{e}{4\pi\epsilon_0} \frac{Q}{r^3} \vec{r}, \quad (4.14)$$

где r – расстояние от неподвижного заряженного центра до движущегося заряда; ϵ_0 – электрическая постоянная.

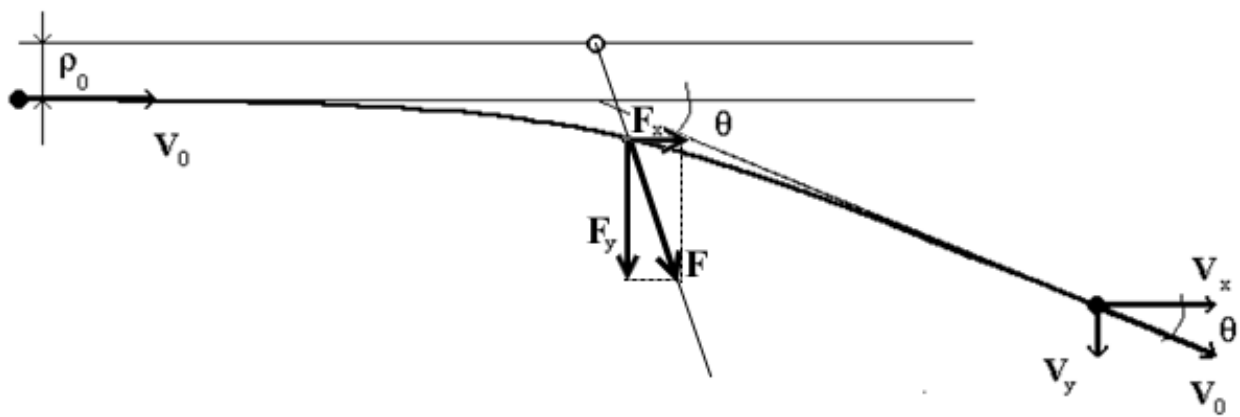


Рис. 4.3. Траектория движения электрического заряда в поле рассеивающего центра. Обозначены направления проекций силы, действующей на заряд в некоторой точке траектории, и скорости заряда на большом удалении от рассеивающего центра

В расчёте проекций ускорения используем проекции силы $\vec{F}$:

$$\begin{cases} F_x = \frac{1}{4\pi\epsilon_0} \frac{eQ}{r^3} r_x, \\ F_y = \frac{1}{4\pi\epsilon_0} \frac{eQ}{r^3} r_y. \end{cases} \quad (4.15)$$

Разделим обе части уравнения на массу частицы и обозначим $\frac{1}{4\pi\epsilon_0 m}$ за k . Тогда из второго закона Ньютона $\frac{dV}{dt} = \frac{F}{m}$:

$$\begin{cases} \frac{dV_x}{dt} = k \frac{eQ}{r^3} r_x, \\ \frac{dV_y}{dt} = k \frac{eQ}{r^3} r_y. \end{cases} \quad (4.16)$$

Теперь мы можем решать задачу в безразмерных величинах, задав определённое значение параметру k . Например, $k = 1$.

Движение заряда в однородном магнитном поле

При движении заряженной частицы с зарядом e и скоростью V в произвольном магнитном поле на нее действует сила Лоренца

$$\vec{F} = e(\vec{V} \times \vec{B}), \quad (4.17)$$

где $\vec{B}$ – индукция магнитного поля, являющаяся функцией координат и времени.

Система уравнений для нахождения траектории частицы:

$$\begin{cases} \frac{d\vec{r}}{dt} = \vec{V}, \\ \frac{d\vec{V}}{dt} = \frac{e}{m} \vec{V} \times \vec{B}. \end{cases} \quad (4.18)$$

Мы рассматриваем движение частицы в плоскости (X,Y) , перпендикулярной вектору $\vec{B}$. Кроме того, считаем что направление вектора магнитной индукции не меняется в пространстве. Тогда система уравнений движения будет иметь вид:

$$\begin{cases} \frac{dx}{dt} = V_x, \\ \frac{dy}{dt} = V_y; \end{cases} \quad \begin{cases} \frac{dV_x}{dt} = \frac{eB}{m} V_y, \\ \frac{dV_y}{dt} = -\frac{eB}{m} V_x. \end{cases} \quad (4.19)$$

Моделирование движения системы небесных тел

К небесным телам относятся планеты, астероиды, кометы, метеориты, звёзды и прочие астрономические объекты. Рассмотрим модель системы планет, вращающихся вокруг некоторой звезды. Моделирование произведём на примере Солнечной системы.

Параметры модели:

- Солнечную систему будем рассматривать как Солнце и вращающиеся вокруг него восемь планет, а также астероид Плутон. Каждое из этих небесных тел имеет свою орбиту, свою скорость, массу и т. д.
- размеры планет будут отрисовываться не в масштабе, но так, чтобы все планеты были различимы.
- основные параметры движения планет приведены в табл. 4.1.

Как гласит первый закон Кеплера траектория движения планет вокруг Солнца представляет собой эллипс, в одном из фокусов которого расположено Солнце (рис. 4.4).

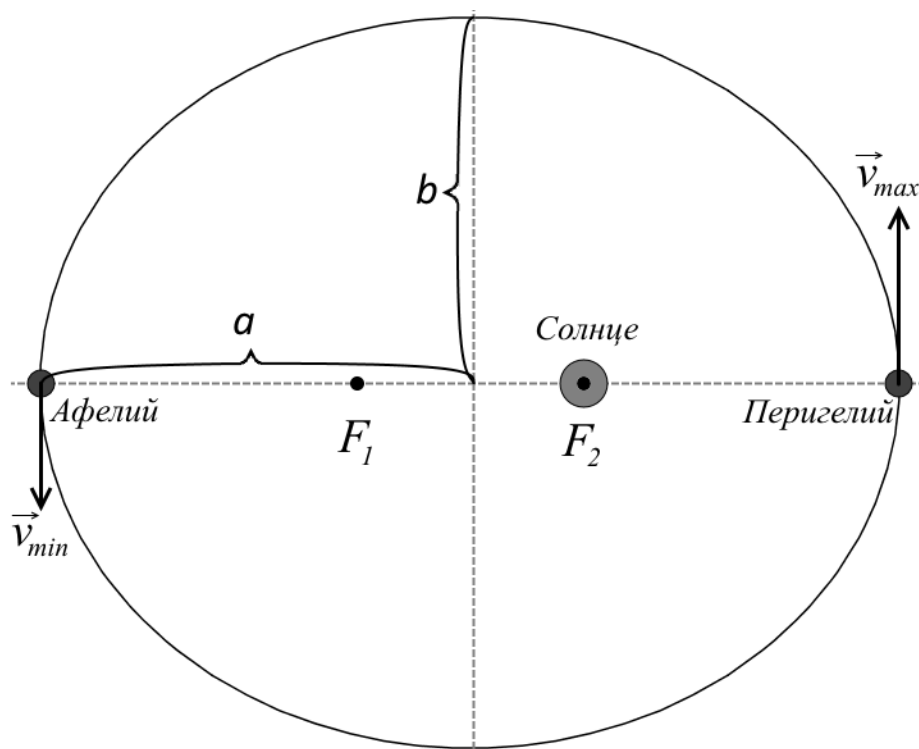


Рис. 4.4. Схематическое изображение орбиты планеты, движущейся вокруг Солнца

Таблица 4.1. Некоторые физические параметры планет Солнечной системы

Параметр	Меркурий	Венера	Земля	Марс
Среднее расстояние от Солнца, а.е.	0,387	0,723	1,000	1,524
Расстояние от Солнца в перигелии, а.е.	0,308	0,718	0,983	1,38
Максимальная орбитальная скорость, км/с	58,98	35,26	30,29	26,50
Расстояние от Солнца в афелии, а.е.	0,467	0,728	1,017	1,67
Минимальная орбитальная скорость, км/с	38,86	34,79	29,29	21,97
Сидерический период обращения, сут	88,0	224,7	365,3	687,0
Масса, кг	3,28e23	4,88e24	5,98e24	6,40e23
Масса (в единицах земной массы)	0,005	0,816	1,000	0,107
Средний радиус, км	2440	6052	6367	3390

Параметр	Юпитер	Сатурн	Уран	Нептун	Плутон
Среднее расстояние от Солнца, а.е.	5,20	9,54	19,18	30,07	39,44
Расстояние от Солнца в перигелии, а.е.	4,95	9,02	18,29	29,81	29,66
Максимальная орбитальная скорость, км/с	13,72	10,18	7,11	5,5	6,1
Расстояние от Солнца в афелии, а.е.	5,46	10,05	20,10	30,33	49,31
Минимальная орбитальная скорость, км/с	12,44	9,09	6,49	5,37	3,71
Сидерический период обращения, лет	11,86	29,46	84,01	164,8	247,7
Масса, кг	1,90e27	5,68e26	8,70e25	1,03e26	1,0e22
Масса (в единицах земной массы)	318	95,1	14,6	17,2	0,0017
Средний радиус, км	69170	57315	25265	24550	1151

Согласно второму закону Кеплера радиус-вектор, соединяющий Солнце и планету, за равные промежутки времени описывает равные площади. Из этого закона следует, что планета имеет различные скорости в разных точках орбиты. Соответственно в ближайшей к Солнцу точке орбиты (*перигелий*) она имеет максимальную линейную скорость, а в наиболее удалённой точке орбиты (*афелий*) – минимальную.

Также для планет и их спутников справедлив третий закон Кеплера: квадраты периодов обращения планет вокруг Солнца относятся, как кубы больших полуосей орбит планет:

$$\frac{T_1^2}{T_2^2} = \frac{a_1^3}{a_2^3}, \quad (4.20)$$

где T_1 и T_2 – периоды обращения двух планет вокруг Солнца; a_1 и a_2 – длины больших полуосей их орбит (см. рис. 4.4).

Движение планет вокруг Солнца подчиняется закону всемирного тяготения Ньютона. Он гласит, что сила гравитационного притяжения между двумя телами массы M и m , находящимися на расстоянии R друг от друга, равна

$$\vec{F} = G \frac{Mm}{R^3} \vec{R}, \quad (4.21)$$

где G – гравитационная постоянная, равная $6,67384(80) \cdot 10^{-11} \text{ м}^3/(\text{кг} \cdot \text{с}^2)$, $\vec{R}$ – радиус-вектор, соединяющий планету и Солнце.

Алгоритм расчета траектории движения планеты полностью соответствует алгоритму, приведённому в начале параграфа. Следуя описанному методу, получим вид системы (4.1) для данной задачи.

Из второго закона Ньютона ускорение планеты:

$$\frac{d\vec{V}}{dt} = GM \frac{\vec{R}}{R^3}, \quad (4.22)$$

Тогда в проекциях на оси декартовой системы координат (4.1) примет вид:

$$\begin{cases} \frac{dx}{dt} = V_x, \\ \frac{dy}{dt} = V_y; \\ \frac{dV_x}{dt} = GM \frac{R_x}{R^3}, \\ \frac{dV_y}{dt} = GM \frac{R_y}{R^3}. \end{cases} \quad (4.23)$$

Задания для самостоятельного выполнения

Лабораторная работа 3

Тема: Расчёт траектории движения тела в поле сил

Задание: Используя метод Эйлера, создать компьютерную модель движения заданного физического тела в некотором поле сил. Изобразить анимацию процесса и траекторию движения тела.

Указание: Пример создания модели равномерного движения тела с помощью библиотеки Qt приведён в прил. II.

Вариант 1

Дано сферическое тело, которое движется в поле силы тяжести. Учесть силу трения воздуха. Положить её пропорциональной квадрату скорости движения тела в воздушном потоке.

Входные данные: масса тела, начальная скорость, направление начальной скорости (угол), скорость ветра, коэффициент сопротивления тела о воздух.

Вариант 2

Дан электрический заряд, который движется в поле неподвижного одноимённо заряженного центра.

Входные данные: начальная скорость частицы, прицельный параметр, величина заряда рассеивающего центра (положение на форме фиксировано), величина рассеиваемого заряда.

Вариант 3

Дан электрический заряд, который движется в однородном магнитном поле перпендикулярно направлению вектора магнитной индукции. Предоставить пользователю возможность «включать» и «выключать» поле.

Входные данные: начальная скорость частицы, направление начальной скорости, величина вектора магнитной индукции, масса и величина заряда.

Лабораторная работа 4

Тема: Расчет траектории движения системы небесных тел

Задание:

1. Создать модель движения планет Солнечной системы вокруг Солнца по закону всемирного тяготения, реализовать анимацию.
2. Определить период обращения заданной планеты вокруг Солнца как время, за которое она возвращается в начальное положение.
3. Проверить выполнение третьего закона Кеплера.

Указания:

Положение планет удобно задавать относительно Солнца. За начало координат в таком случае необходимо принять положение Солнца.

В справочной литературе чаще всего приводятся только средние значения величин удаления планет от Солнца, линейных скоростей относительно него. Такие данные не годятся для построения реалистичной модели. В некоторых источниках приводятся также максимальные и минимальные значения этих величин (см. табл. 4.1), которые удобно брать в качестве начальных условий задачи. В частности, в качестве начальных координат планеты можно взять такие, которые соответствовали бы её нахождению в афелии (или перигелии). Тогда в качестве начальной скорости планеты необходимо брать минимальную (или максимальную) орбитальную скорость.

В данной работе также необходимо будет выводить на экран изображение планет с заданным шагом времени. Для того чтобы анимация была плавной, достаточно выставить временной интервал $dt = 20$ мс. Через этот промежуток времени кадры анимации будут сменять друг друга. Для возможности пронаблюдать движение планет необходимо задать «масштаб» времени, в котором одна секунда экранного времени будет соответствовать значению расчётного не менее десяти суток (рекомендуемое значение: от 10 до 1000).

Пример создания простейшей модели солнечной системы с помощью *Qt Creator* см. в прил. III.

§ 5. Математическое моделирование колебательных движений

Колебательные процессы – одно из самых распространенных в природе явлений. В таких процессах некоторая физическая величина через определенные промежутки времени принимает те же самые (или близкие) значения. Такой колеблющейся величиной может быть координата и скорость частицы, заряд кон-

денсатора, напряженность электрического поля и т. д. Если периодически меняющейся величиной является положение тела (частицы) в пространстве, то мы имеем дело с механическими колебаниями (колебательным движением).

Линейные колебания

Рассмотрим простейшую колебательную систему (линейный осциллятор), представляющую собой тело массы m , совершающее одномерное движение вдоль оси X под действием силы $F(x)$ вблизи положения равновесия $x = 0$, так что $F(x = 0) = 0$. Считая отклонение от положения равновесия x малым, разложим силу в окрестности точки $x = 0$:

$$F(x) = F(0) + kx + \dots$$

Оставим в разложении только линейный член. Полагаем также, что коэффициент k положителен, а сила $F(x)$ направлена в сторону, противоположную отклонению, т. е. к точке равновесия (в этом случае равновесие устойчивое): $F(x) \approx -kx$. Примером такой системы может служить грузик, движущийся без трения на горизонтальной плоскости под действием силы натяжения пружины.

Уравнение движения тела имеет вид

$$m \frac{dV}{dt} = F(x) = -k(x), \quad (5.1)$$

где скорость V – очевидно производная от перемещения тела $x(t)$. Это определение даёт нам второе уравнение движения тела:

$$V = \frac{dx}{dt}. \quad (5.2)$$

Для определения зависимости отклонения тела $x(t)$ последние два уравнения решаются с некоторыми начальными условиями

$$x(0) = x_0; \quad V(0) = V_0.$$

Введя обозначение $\omega_0^2 = k/m$, из уравнений движения (5.1 – 5.2) получим

$$\frac{d^2x}{dt^2} + \omega_0^2 x = 0, \quad (5.3)$$

решение которого даёт искомую функцию $x(t)$. Это уравнение называется уравнением гармонических колебаний.

При рассмотрении более общих случаев, когда кроме силы F на тело действует также сила трения $F_{\text{тр}}$ (например, за счет сопротивления воздуха или трения о поверхность) и некоторая внешняя сила $F_{\text{вн}}(t)$ (которая может быть произвольной функцией времени), также направленные по оси X , необходимо переписать второй закон Ньютона (уравнение движения (5.1)) с учётом этих сил:

$$m \frac{dV}{dt} = F(x) = -k(x) - F_{\text{тр}}(x) + F_{\text{вн}}(t). \quad (5.4)$$

Полезной характеристикой колебательного движения является зависимость полной энергии тела $E = \frac{mV^2}{2} + U(x)$ от времени. Для силы вида $F(x) = -kx$ потенциальная энергия $U(x) = \frac{kx^2}{2}$. В отсутствие внешних сил и трения ($F_{\text{вн}} = 0$, $F_{\text{тр}} = 0$) полная энергия сохраняется: $E = E_0 = \text{const}$.

Ещё одна характеристика – так называемый фазовый портрет, т. е. зависимость V от w_0x . Иногда под ней также понимают и зависимость V лишь от смещения тела относительно положения равновесия x . Можно заметить, что в случае гармонических колебаний из вышеупомянутого закона сохранения энергии следует соотношение для скорости $V^2 + \omega_0^2 x^2 = V_0^2$. Это значит, что при гармонических колебаниях фазовый портрет представляет собой окружность.

Таким образом, данные характеристики могут служить критериями точности численного решения системы.

Алгоритм расчета траектории движения тела:

1. *Задаются начальные условия – координата и скорость (x_0, V_0) .*
2. *Определяются силы, действующие на тело.*
3. Путём решения уравнений движения *методом Эйлера* с заданным шагом dt по времени *определяются:*
 - 3.1. *Значения скорости тела в данной точке*
 - 3.2. *Координата тела в следующей точке x_{i+1}*

Шаги 1-3 повторяются по циклу. На каждой итерации определяется новое положение тела.

Математический маятник

Рассмотрим теперь в качестве колебательной системы математический маятник на стержне длиной R . Если отклонить его от положения равновесия, или в этом положении сообщить ему некоторую начальную скорость, то тело придет в движение (рис. 5.1).

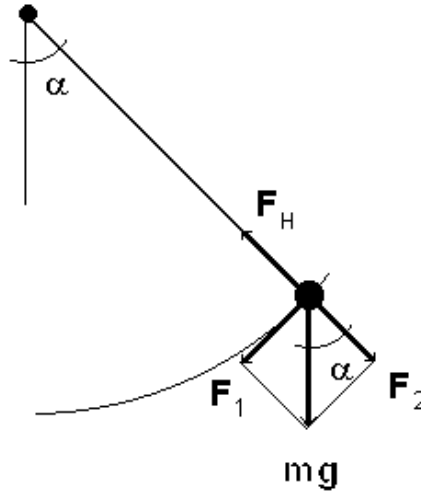


Рис. 5.1. Математический маятник

Так как составляющая силы тяжести $F_2 = mg \cos \alpha$ уравнивается натяжением стержня, то тело движется только под действием составляющей $F_1 = mg \sin \alpha$ по дуге окружности радиуса R . В любой момент времени сила $\vec{F}_1$ и скорость $\vec{V}$ направлены по касательной, так что движение является одномерным и характеризуется только одним параметром – углом α , отсчитываемым от положения равновесия, который в данном случае можно принять за обобщенную координату. Уравнения, описывающие движение тела по окружности, имеют вид

$$\frac{d\alpha}{dt} = \Omega, \quad (5.5)$$

$$mR^2 \frac{d\Omega}{dt} = -(mg \sin \alpha)R, \quad (5.6)$$

где первое уравнение определяет угловую скорость Ω , второе – основное уравнение динамики вращательного движения тела с моментом инерции mR^2 . $F_1 R = (mg \sin \alpha)R$ – момент сил. Знак "–" перед ним означает, что сила $\vec{F}_1$ возвращает тело к положению равновесия.

Рассмотрим более общий случай, когда кроме силы $\vec{F}_1$ на тело действует также сила трения вида $F_{\text{тр}} = -\beta V$. Добавляя момент этой силы в правую часть основного уравнения динамики (5.6) и вводя обозначение $\omega_0^2 = g/R$, получим

$$\frac{d\Omega}{dt} = -\omega_0^2 \sin \alpha - \frac{\beta}{m} \Omega. \quad (5.7)$$

Для решения задачи на нахождение траектории маятника к уравнениям движения необходимо добавить начальные условия

$$\alpha(0) = \alpha_0; \quad \Omega(0) = \Omega_0.$$

Из (5.5) и (5.7) можно получить уравнение

$$\frac{d^2\alpha}{dt^2} + \frac{\beta}{m} \frac{d\alpha}{dt} + \omega_0^2 \sin \alpha = 0. \quad (5.8)$$

Это уравнение описывает затухающие колебания математического маятника.

В случае малого отклонения от положения равновесия $\alpha \ll 1$ можно заметить $\sin \alpha \approx \alpha$. В таком случае движение тела происходит практически по прямой линии вдоль оси X .

При свободных колебаниях математического маятника сохраняется полная энергия $E = \frac{mV^2}{2} + mgh$. В переменных α и $\Omega = \frac{d\alpha}{dt}$ выражение для неё имеет вид

$$E = mR^2 \left(\frac{\Omega^2}{2} + \omega_0^2 (1 - \cos \alpha) \right). \quad (5.9)$$

При численном решении уравнений движения можно воспользоваться указанным в предыдущем разделе алгоритмом, где роль координаты x будет играть угол α , а скорости V – угловая скорость Ω .

Задания для самостоятельного выполнения

Лабораторная работа 5

Тема: Моделирование колебательного движения

Задание:

1. Исследовать колебательный процесс с учётом сил трения. Реализовать анимацию процесса (простейший маятник).

2. Определить период движения колеблющегося тела как время, за которое оно возвращается в начальное положение
3. Добавить компонент для вывода графика зависимости отклонения тела со времени от положения равновесия $x(t)$.
4. Добавить компонент для вывода графика зависимости, указанного в табл. 5.1.

Таблица 5.1. Варианты для выполнения лабораторной работы 2

Вар.	Маятник*	Дополнительное задание
1	М	Построить <i>фазовый портрет</i> (график, изображающий точки в координатах (V, x)), убедиться, что в отсутствии трения (гармонические колебания) график представляет собой окружность
2	Л	
3	М	Построить график зависимости <i>полной энергии от времени</i> $E(t)$, убедиться, что в отсутствии трения (гармонические колебания) график представляет собой постоянную функцию
4	Л	
5	М	На одном графике построить зависимости <i>кинетической и потенциальной энергий от времени</i> ($E_k(t)$ и $E_p(t)$), убедиться, что в отсутствии трения их максимальные значения совпадают
6	Л	

* М – математический, Л – линейный.

§ 6. Моделирование векторных полей

Векторное поле – область пространства, в каждой точке которой можно определить некоторый вектор $\vec{A}(\vec{r})$. К векторным полям относятся силовые поля, электростатические поля, поля скоростей для жидкости и т. д. Под вектором $\vec{A}$ подразумевается соответствующая векторная величина в зависимости от того, какое поле рассматривается.

Моделирование векторных полей обычно сводится к двум задаче визуального изображения силового поля в виде карты линий поля или силовых линий. Наибольшей наглядностью при этом обладают двумерные поля или поля, обла-

дающие аксиальной симметрией, для которых можно построить двумерные карты линий поля. Будем далее рассматривать именно такие поля, т. е. ограничимся двумерной задачей, будем строить силовые линии на плоскости.

Силовые линии

Силовая линия – это линия, касательная к которой в каждой точке совпадает по направлению с вектором $\vec{A}$. Согласно этому определению, можно записать уравнения силовой линии:

$$\frac{dx}{dl} = \frac{A_x}{|A|}, \quad \frac{dy}{dl} = \frac{A_y}{|A|}. \quad (6.1)$$

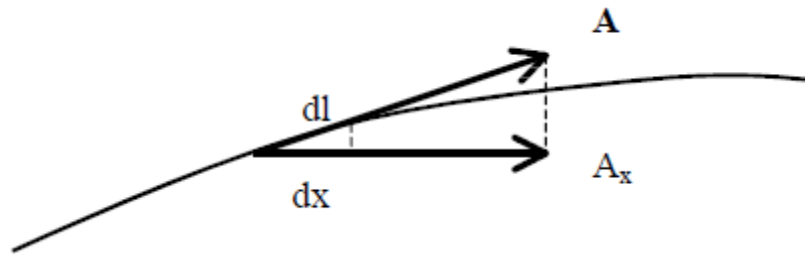


Рис. 6.1. Силовая линия векторного поля

Если вектор $\vec{A}$ в каждой точке выделенной области определен однозначно, то силовые линии нигде не пересекаются в этой области.

Построение силовой линии сводится к решению системы дифференциальных уравнений первого порядка (6.1) с заданными начальными условиями. В качестве начальных условий можно выбрать координаты некоторой точки пространства (x_0, y_0) . Выбор этой точки произволен, так как через любую точку пространства можно провести силовую линию. Будем называть эту точку начальной. Решением вышеприведённых уравнений является уравнение силовой линии:

$$f(x, y) = f(x_0, y_0) = 0.$$

Карта силовых линий в некоторой области поля представляет собой семейство силовых линий с разными значениями $f(x_0, y_0)$. Необходимо подбирать некоторые правила выбора начальных точек для разных силовых линий. Это делается для того, чтобы модель правильно изображала поведение поля в пространстве, например, чтобы густота силовых линий соответствовала величине вектора $\vec{A}$.

Алгоритм построения силовой линии

1. *Задаются координаты начальной точки (x_0, y_0) .*
2. *С помощью физических законов, описывающих векторное поле, определяются значения A_x, A_y в этой точке.*
3. *Путём решения (6.1) для $\vec{A}$ численными методами с заданным шагом определяются координаты следующей точки силовой линии (x_1, y_1) .*

Шаги 1-3 повторяются для каждой вновь определённой точки, вычисляются координаты следующей точки.

Начальная точка и вся силовая линия должны лежать внутри выделенной области пространства, где выполняется условие (4.2). Особые точки пространства, где расположены источники поля (электрические заряды для электрического поля, проводники с током для магнитного поля и т. д.), должны быть исключены из этой области пространства. Подробно эта процедура описана в следующих параграфах пособия. Завершается построение силовой линии, когда она выходит за пределы выделенной области пространства.

Силовые линии электрического поля

Источником электрического поля являются заряженные тела. В качестве модельной задачи рассмотрим построение силовых линий поля системы точечных зарядов. Электрическое поле характеризуется вектором напряженности E . Согласно уравнениям Максвелла

$$\operatorname{div} \vec{E} = \frac{\rho}{\varepsilon_0}, \quad (6.2)$$

где ρ – объемная плотность электрических зарядов в точке наблюдения; ε_0 – электрическая постоянная. В областях пространства, где электрические заряды отсутствуют ($\rho = 0$), получим

$$\operatorname{div} \vec{E} = 0. \quad (6.3)$$

Будем рассматривать поля, создаваемые точечными электрическими зарядами. Для системы N точечных зарядов величина $\vec{E}$ определяется из закона Кулона:

$$\vec{E} = \sum_{i=1}^N \vec{E}_i = \sum_{i=1}^N \frac{q_i}{4\pi\epsilon_0} \frac{\vec{r}_i}{r_i^3}, \quad (6.4)$$

где $\vec{E}_i$ – напряженность поля, создаваемого i -м точечным зарядом в точке наблюдения; $\vec{r}_i$ – радиус-вектор, проведенный из i -го заряда в точку наблюдения; q_i – его величина; ϵ_0 – электрическая постоянная.

Учитывая, что $\vec{r}_i = \vec{r} - \vec{R}_i$, где $\vec{r}$ – радиус-вектор точки наблюдения, а $\vec{R}_i$ – радиус-вектор i -ого заряда и, расписывая (6.4) по координатам, в двухмерном случае получим

$$\begin{cases} E_x = \sum_{i=1}^N E_{x_i} = \sum_{i=1}^N \frac{q_i}{4\pi\epsilon_0} \frac{x - x_i}{(\sqrt{(x - x_i)^2 + (y - y_i)^2})^2}, \\ E_y = \sum_{i=1}^N E_{y_i} = \sum_{i=1}^N \frac{q_i}{4\pi\epsilon_0} \frac{y - y_i}{(\sqrt{(x - x_i)^2 + (y - y_i)^2})^2}. \end{cases} \quad (6.5)$$

$$E = \sqrt{E_x^2 + E_y^2}. \quad (6.6)$$

Здесь x_i, y_i, z_i – координаты i -го точечного заряда, x, y, z – координаты точки наблюдения.

Для построения силовых линий в каждой точке используются отношения проекций поля к длине вектора, т. е. используются величины $E_x/E, E_y/E$ и поэтому в выражении (6.4) можно положить множитель $1/4\pi\epsilon_0$ равным единице, а величину заряда брать не в единице измерения СИ, а в условных единицах. Например, за единицу такого заряда брать величину самого маленького из зарядов системы.

Отметим, что для построения силовых линий в выделенной области пространства в ней не должны находиться источники поля. Если точечный заряд находится внутри этой области, то силовая линия должна прерваться в некоторой малой окрестности заряда, т. к. вблизи заряда напряженность электрического поля неограниченно возрастает. Таким образом, окрестность точечного заряда исключается из области пространства, в которой строятся силовые линии. Условие кон-

ца силовой линии, таким образом, означает, что линия подошла к границе выделенной области пространства или к точечному заряду.

Выбор начальных точек для карты силовых линий

Для построения карты силовых линий необходимо задать алгоритм выбора начальных точек для линий. Для этого можно воспользоваться тем, что вблизи точечного заряда его собственное поле значительно превосходит по величине поля остальных зарядов. Поэтому картина силовых линий вблизи точечного заряда практически не отличается от картины силовых линий одиночного заряда.

Учитывая, что силовые линии точечного заряда расходятся в виде лучей симметрично относительно точки расположения заряда (напряженность электрического поля точечного заряда зависит только от расстояния до него), в качестве начальных точек для силовых линий можно выбрать точки окружности малого радиуса ρ , проведенной вокруг заряда, расположенные равномерно через угол $\alpha = \rho/N$, где N – число силовых линий (рис. 6.2).

Для начальной точки j -й силовой линии таким образом имеем

$$\begin{cases} x_i = x_0 + \rho \cos j\alpha, \\ y_i = y_0 + \rho \sin j\alpha. \end{cases} \quad (6.7)$$

Здесь x_0, y_0 – координаты точечного заряда.

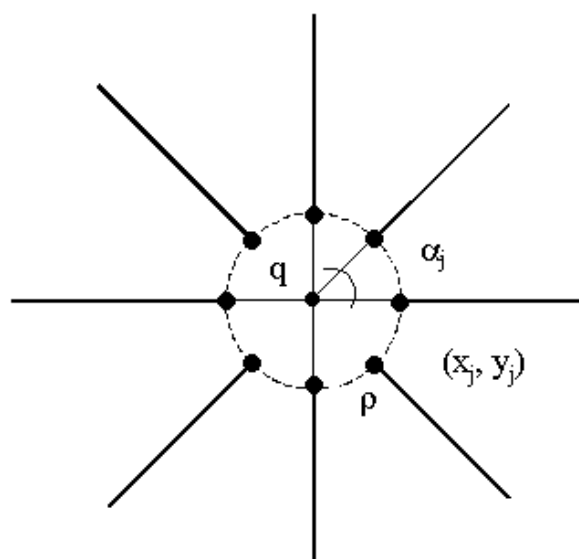


Рис. 6.2. Пример выбора начальных точек для карты силовых линий поля точечного заряда

При таком выборе начальных точек исключается из рассмотрения область вокруг особой точки – места расположения точечного заряда, где $E \rightarrow \infty$. После выбора начальной точки (6.7) строится силовая линия согласно описанному алгоритму.

Если имеется несколько электрических зарядов, подобный способ позволяет построить набор силовых линий, исходящих из каждого заряда. Так как одна и та же силовая линия начинается на одном заряде и заканчивается на другом, возникают проблемы совместимости систем силовых линий, исходящих из разных зарядов.

Силовые линии магнитного поля

Задача построения силовой линии магнитного поля (линии магнитной индукции) аналогична задаче построения силовой линии электрического поля, так как для магнитного поля тоже справедливо условие

$$\operatorname{div} \vec{B} = 0. \quad (6.8)$$

Величина $\vec{B}$ определяется из уравнения Максвелла для постоянного магнитного поля в вакууме:

$$\operatorname{rot} \vec{B} = \mu_0 \vec{j}, \quad (6.9)$$

где $\vec{j}$ – плотность тока в данной точке пространства; μ_0 – магнитная постоянная.

Источником магнитного поля является система электрических токов. Из-за отсутствия магнитных зарядов, которые были бы аналогичны точечным электрическим, магнитное поле – вихревое, в отличие от электрического. Наиболее простая конфигурация замкнутой цепи с постоянным током – кольцо. Поэтому кольцо с током можно считать источником стационарного магнитного поля. Кольцо имеет в отличие от точечного заряда конечные размеры. Поэтому для определения магнитной индукции $\vec{B}$ кольца с током в произвольной точке пространства используют закон Био-Савара.

Разобьем кольцо на элементы с током $d\vec{l}$. Магнитная индукция от такого элемента тока по закону Био-Савара

$$d\vec{B} = \frac{\mu_0 I [d\vec{l} \times \vec{r}]}{4\pi r^3}. \quad (6.10)$$

Здесь I – ток, протекающий в контуре; $d\vec{l}$ – элемент контура, направление вектора $d\vec{l}$ совпадает с направлением тока, $\vec{r}$ – радиус-вектор, проведенный из точки расположения элемента тока в точку наблюдения (рис. 6.3).

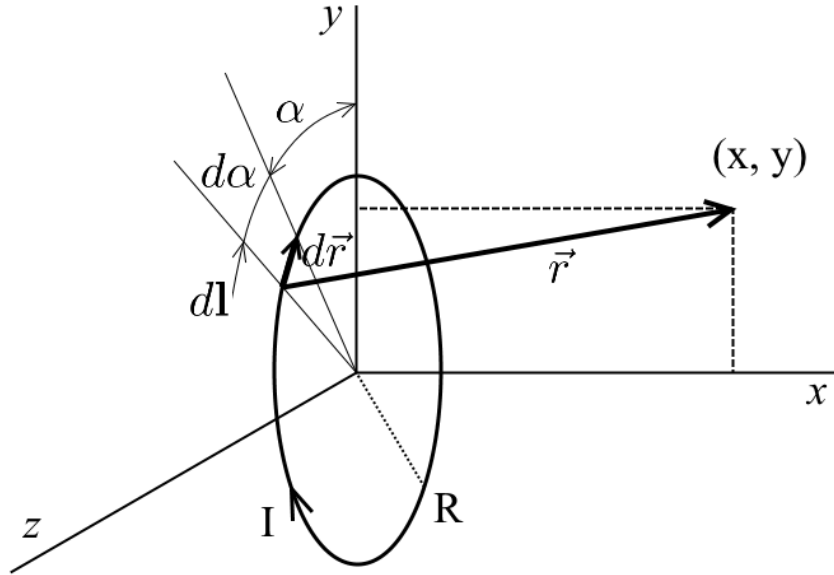


Рис. 6.3. Определение магнитной индукции кругового тока в плоскости, перпендикулярной плоскости проводника.

Будем решать задачу в безразмерных величинах. Тогда постоянный множитель $\frac{\mu_0}{4\pi}$ можно положить равным единице. Также условимся далее писать уравнения для индукции магнитного поля в тех же обозначения, что и раньше, однако имея в виду безразмерные величины.

Суммарное поле $\vec{B}$ определяется интегрированием выражения

$$d\vec{B} = \frac{I [d\vec{l} \times \vec{r}]}{r^3} \quad (6.11)$$

по контуру с током. За ось X примем ось симметрии, проходящую через центр кольца перпендикулярно его плоскости. Плоскость (X, Y) проведём через точку наблюдения и ось симметрии. Компоненты магнитного поля $d\vec{B}$ от элемента кольца с током $d\vec{l}$ определяются выражениями

$$\begin{cases} dB_x = \frac{IR^2}{r^3} d\alpha - \frac{IyR \cos \alpha}{r^3} d\alpha, \\ dB_y = \frac{IxR \cos \alpha}{r^3} d\alpha, \\ dB_z = \frac{IxR \sin \alpha}{r^3} d\alpha. \end{cases} \quad (6.12)$$

Здесь R – радиус кольца с током; r – расстояние от точки наблюдения до элемента с током $d\vec{l}$; α – азимутальный угол, на который повернут элемент с током $d\vec{l}$ относительно плоскости (X, Y) .

Для нахождения суммарного поля от кольца надо проинтегрировать выражения из системы (6.12) по азимутальному углу α от 0 до 2π . При этом компонент поля B_z обратится в 0 в силу аксиальной симметрии. Для оставшихся компонент получим

$$\begin{cases} B_x = IR^2 \int_0^{2\pi} \frac{1}{r^3} d\alpha - IyR \int_0^{2\pi} \frac{\cos \alpha}{r^3} d\alpha, \\ B_y = IxR \int_0^{2\pi} \frac{\cos \alpha}{r^3} d\alpha. \end{cases} \quad (6.13)$$

Последняя система определяет магнитную индукцию кольца с током в произвольной точке плоскости (X, Y) . Для вычисления интегралов можно использовать метод трапеций или метод Симпсона.

При построении силовых линий магнитного поля надо учесть, что они замкнуты вокруг токов, их вызывающих. Поэтому необходимо выбирать более точный алгоритм численного решения системы, например метод Рунге-Кутты. Более сложную систему токов, соленоид, можно рассматривать как набор колец, плотно прижатых друг к другу. Для вычисления магнитной индукции необходимо просуммировать в каждой точке поля, создаваемые каждым кольцом в отдельности.

Построение карты силовых линий. Для выбора начальных точек для карты силовых линий можно использовать симметрию задачи, в частности то, что силовые линии перпендикулярны плоскости кольца с током (это является следствием того, что картина силовых линий симметрична относительно плоскости кольца). Начальные точки для силовых линий при этом надо выбирать на линии, являющейся проекцией кольца с током на плоскость экрана (рис. 6.4).

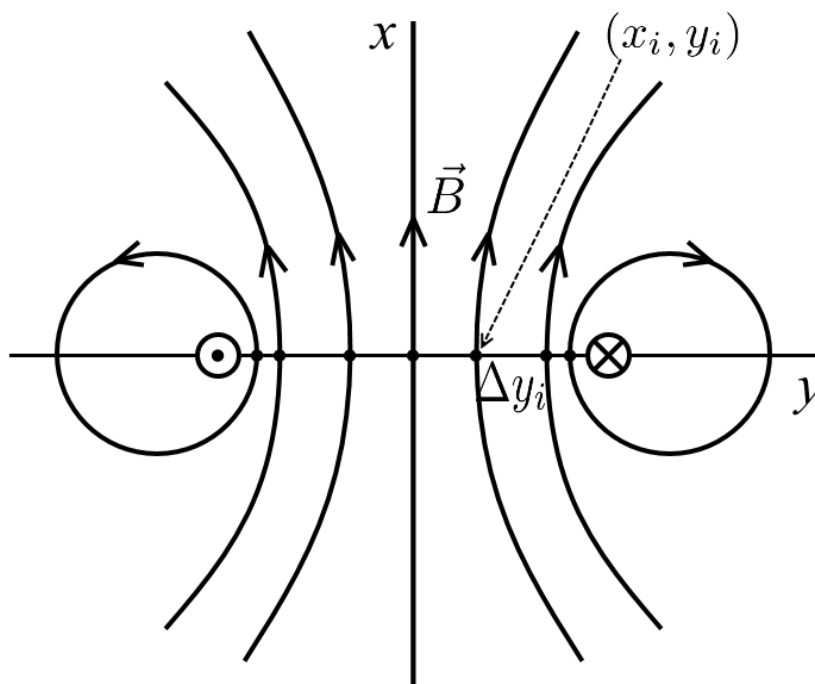


Рис. 6.4. Пример выбора начальных точек для построения силовых линий магнитного поля

Расстояние между соседними линиями должно быть обратно пропорционально величине B , т. е.

$$\Delta y = \frac{c}{B(y)},$$

где c – константа, координата y отсчитывается от оси. При этом первая силовая линия находится на оси, остальные распределены симметрично относительно оси. Область вблизи кольца с током должна быть исключена, т. к. это особая точка и поле в ней неограниченно растёт.

Задания для самостоятельного выполнения

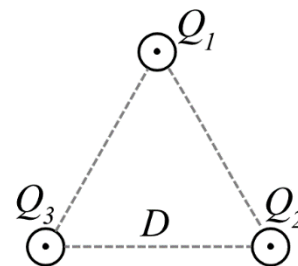
Лабораторная работа 6

Тема: Моделирование электростатического поля системы зарядов

Задание: Построить карту силовых линий электростатического поля системы зарядов.

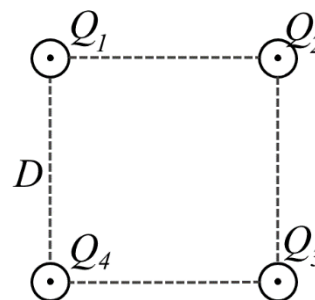
Вариант 1

Система, состоящая из трёх точечных зарядов Q_1, Q_2, Q_3 , расположенных в вершинах равностороннего треугольника со стороной D . Рассмотреть случай одноимённых и разноимённых зарядов. Исследовать поле на больших расстояниях от них, при различных соотношениях величины зарядов.



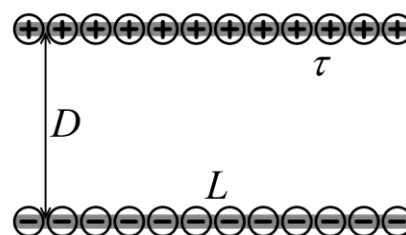
Вариант 2

Система, состоящая из четырёх точечных зарядов Q_1, Q_2, Q_3 и Q_4 , расположенных в вершинах квадрата со стороной D . Рассмотреть случай одноимённых и разноимённых зарядов. Исследовать поле на больших расстояниях от них, при различных соотношениях величины зарядов.



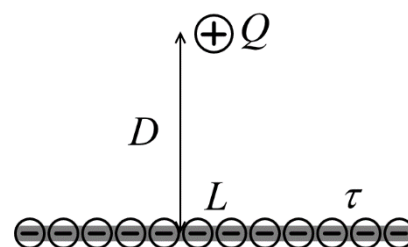
Вариант 3

Плоский конденсатор с длиной пластин L , расстоянием между ними D . Пластины представить как линейку точечных зарядов, расстояние между которыми много меньше, чем длина линейки и расстояния между пластинами. Линейная плотность заряда на каждой пластине τ .



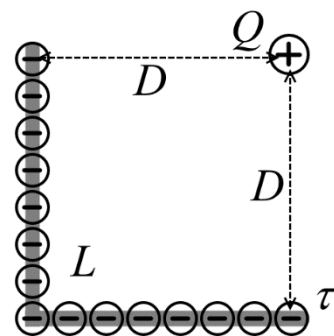
Вариант 4

Заряженная пластина длиной L и противоположно заряженная проволока, расположенная на расстоянии D (цилиндр). Пластина и проволока расположены перпендикулярно плоскости построения. Пластины представить как линейку точечных зарядов, расстояние между которыми много меньше, чем длина линейки. Заряд проволоки Q , линейная плотность заряда на пластине τ .



Вариант 5

Заряженная пластина длиной L согнута пополам под углом 90° , противоположно заряженная проволока находится на расстоянии D от каждого из концов пластины. Пластина и проволока расположены перпендикулярно плоскости построения. Пластину представить как линейку точечных зарядов, расстояние между которыми много меньше, чем длина линейки. Заряд проволоки Q , линейная плотность заряда на пластине τ .



Лабораторная работа 7

Тема: Моделирование магнитного поля системы токов

Задание: Построить карту силовых линий магнитного поля системы электрических токов.

Вариант 1

Система из двух прямолинейных проводников с токами I_1 и I_2 , текущими в одинаковых направлениях. Карту линий строить в плоскости, перпендикулярной этим токам.



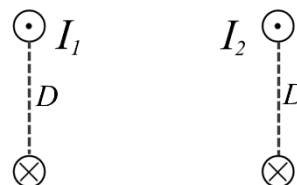
Вариант 2

Система из двух прямолинейных проводников с токами I_1 и I_2 , текущими в противоположных направлениях. Карту линий строить в плоскости, перпендикулярной этим токам.



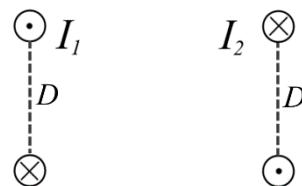
Вариант 3

Система из двух параллельных соосных колец диаметра D с токами I_1 и I_2 , текущими в одинаковых направлениях. Карту линий строить в плоскости, проходящей через центры колец.



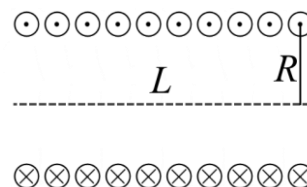
Вариант 4

Система из двух параллельных соосных колец диаметра D с токами I_1 и I_2 , текущими в разных направлениях. Карту линий строить в плоскости, проходящей через центры колец.



Вариант 5

Соленоид конечной длины L радиусом R . Карту линий строить в плоскости, проходящей через центральную ось соленоида.



§ 7. Построение изолиний векторного поля

Изолинии потенциала электростатического поля

Наглядное представление о характере и топологии силового поля дают карты изолиний, т.е. линий равного значения некоторой величины, характеризующей поле. Если силовое поле характеризуется вектором $\vec{A}$, то такой величиной может быть некоторая скалярная функция $f(\vec{A})$. Изолиниями электростатического поля являются эквипотенциальные линии.

Алгоритм построения изолиний не связан с решением дифференциальных уравнений, поэтому здесь не возникает проблемы выбора начальных точек для построения линии. Это позволяет строить карты силовых линий для систем зарядов и токов, где использованный выше метод неприменим. Набор изолиний проводится таким образом, чтобы значение $f(\vec{A})$ на соседних линиях отличалось на одну и ту же величину:

$$\Delta f = f_{i+1}(\vec{A}) - f_i(\vec{A}). \quad (7.1)$$

Необходимо отметить, что данный метод требует вычисления величины $f(\vec{A})$ в каждой точке выделенной области пространства, что может потребовать значительного времени, поэтому этот метод годится для использования в вычислительных машинах с большой производительностью.

Рассмотрим алгоритм построения изолиний на примере эквипотенциальных линий электростатического поля.

Для потенциального поля ($\text{rot } E = 0$) в каждой точке можно ввести потенциал $\varphi(r)$ так, что

$$E = -\text{grad } \varphi. \quad (7.2)$$

Точки, имеющие одинаковый потенциал, образуют эквипотенциальную поверхность. Линия равного потенциала на плоскости является линией пересечения этой плоскостью эквипотенциальной поверхности.

Всегда можно однозначно определить потенциал некоторой точки пространства, приняв, например, за нуль потенциал точки, бесконечно удаленной от системы зарядов. В этом случае потенциал точки с координатами (x, y) для системы точечных зарядов определяется по формуле

$$\varphi(x, y) = \sum \varphi_i = \frac{1}{4\pi\epsilon_0} \sum_{i=1}^N \frac{q_i}{r_i}, \quad (7.3)$$

$$r_i = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2}.$$

Здесь r_i – расстояние от i -го заряда до точки, в которой вычисляется потенциал; x_i, y_i, z_i – координаты i -го заряда; x, y, z – координаты точки наблюдения. Можем положить множитель $1/4\pi\epsilon_0$ равным единице, тогда получим уравнение в безразмерных переменных:

$$\varphi(x, y) = \sum \varphi_i = \sum_{i=1}^N \frac{q_i}{\sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2}}. \quad (7.4)$$

Построение изолиний сводится к определению потенциала всех точек заданной области пространства и выделению тем или иным образом точек с потенциалом, равным заданному. Для построения изолинии необходимо по заданному алгоритму просканировать все точки заданной области пространства.

Для построения карты эквипотенциальных линий необходимо задать значения потенциала на каждой линии. Обычно эквипотенциальные линии проводятся через равные промежутки $\Delta\varphi$, т. е.

$$\varphi_{i+1} = \varphi_i + \Delta\varphi.$$

Заметим, что область вблизи точечного заряда необходимо исключить из рассмотрения, поскольку в этой области значение потенциала бесконечно возрастает.

Наиболее простой метод построения эквипотенциальных линий заключается в окрашивании в определенный цвет области, потенциал в которой лежит в пределах от φ_1 до φ_2 . Для этого достаточно рассчитать потенциал в каждой точке и окрасить ее в нужный цвет. Эквипотенциальными линиями в данном случае будут границы областей, окрашенных разными цветами.

Задания для самостоятельного выполнения

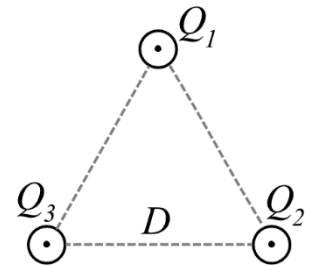
Лабораторная работа 8

Тема: Построение эквипотенциальных линий электростатического поля системы зарядов

Задание: Построить карту эквипотенциальных линий электростатического поля системы зарядов.

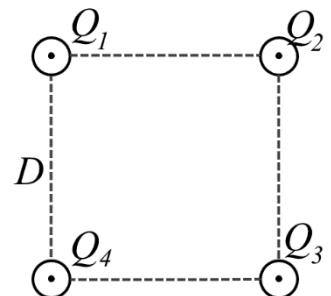
Вариант 1

Система, состоящая из трёх точечных зарядов Q_1, Q_2, Q_3 , расположенных в вершинах равностороннего треугольника со стороной D . Рассмотреть случай одноимённых и разноимённых зарядов. Исследовать поле на больших расстояниях от них, при различных соотношениях величины зарядов.



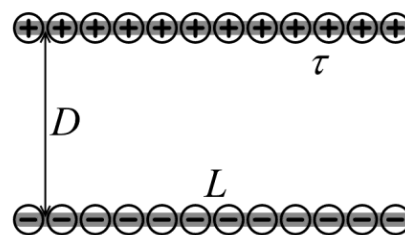
Вариант 2

Система, состоящая из четырёх точечных зарядов Q_1, Q_2, Q_3 и Q_4 , расположенных в вершинах квадрата со стороной D . Рассмотреть случай одноимённых и разноимённых зарядов. Исследовать поле на больших расстояниях от них, при различных соотношениях величины зарядов.



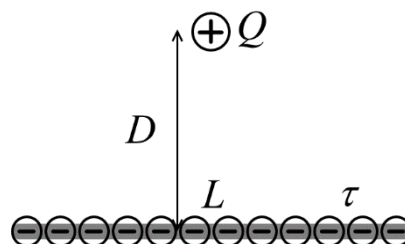
Вариант 3

Плоский конденсатор с длиной пластин L , расстоянием между ними D . Пластины представить как линейку точечных зарядов, расстояние между которыми много меньше, чем длина линейки и расстояния между пластинами. Линейная плотность заряда на каждой пластине τ .



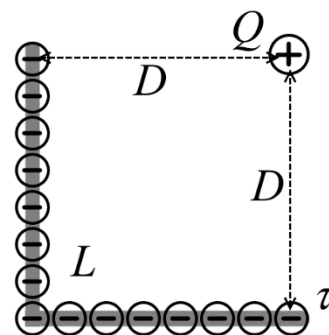
Вариант 4

Заряженная пластина длиной L и противоположно заряженная проволока, расположенная на расстоянии D (цилиндр). Пластина и проволока расположены перпендикулярно плоскости построения. Пластины представить как линейку точечных зарядов, расстояние между которыми много меньше, чем длина линейки. Заряд проволоки Q , линейная плотность заряда на пластине τ .



Вариант 5

Заряженная пластина длиной L согнута пополам под углом 90° , противоположно заряженная проволока находится на расстоянии D от каждого из концов пластины. Пластина и проволока расположены перпендикулярно плоскости построения. Пластины представить как линейку точечных зарядов, расстояние между которыми много меньше, чем длина линейки. Заряд проволоки Q , линейная плотность заряда на пластине τ .



I. Работа с графиками в Qt с использованием библиотеки QCustomPlot

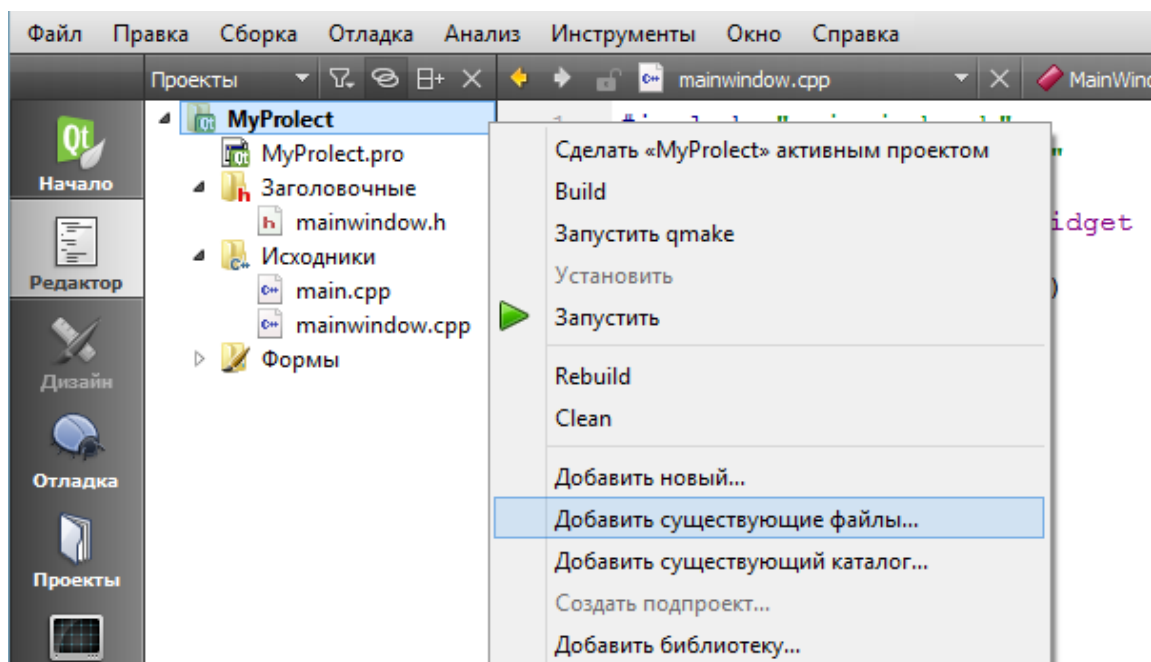
Для простоты работы с графиками в *Qt Creator* удобно использовать специально предназначенные для этого библиотеки, такие как *QWT* или *QCustomPlot*. Эти библиотеки являются свободно распространяемыми, имеют подробную документацию. Рассмотрим пример создания простого приложения для работы с графиками с использованием библиотеки *QCustomPlot*¹.

1. Создадим проект в среде *Qt Creator*.

Файл → Новый файл или проект → Приложение → Приложение Qt Widgets.

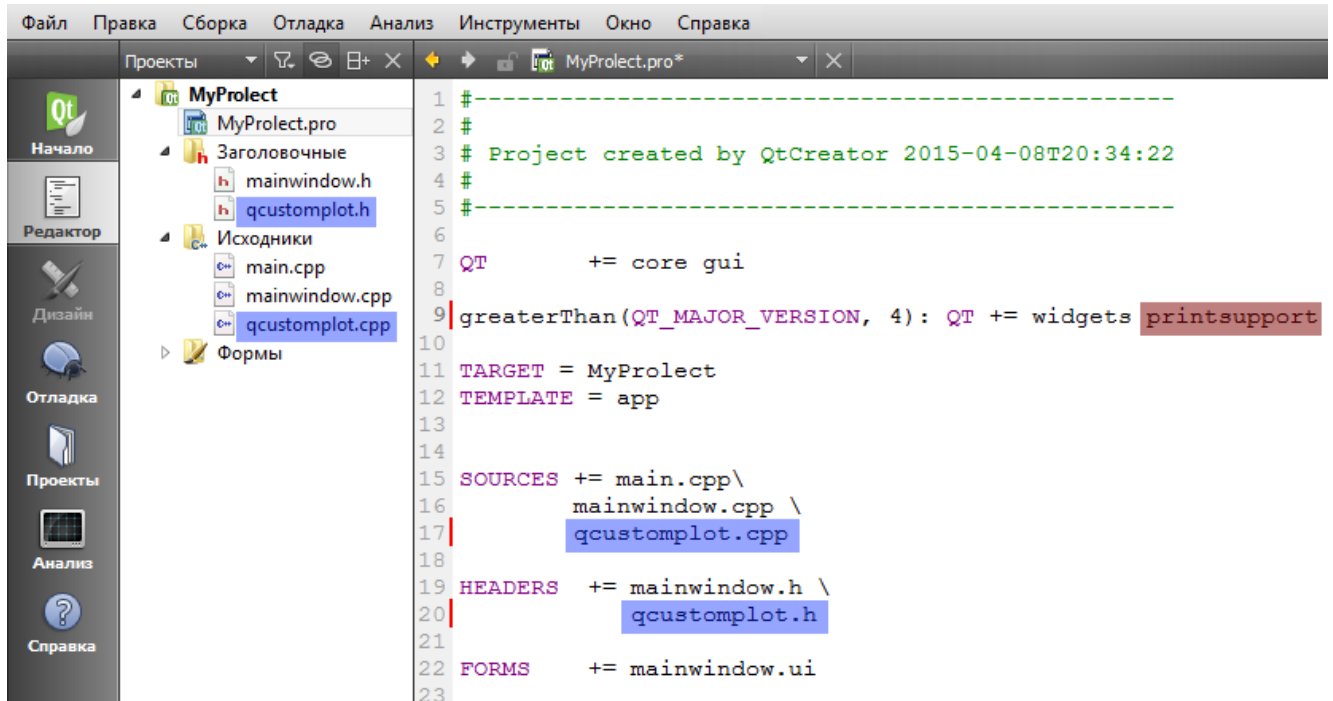
Далее следуя инструкциям на экране введём название проекта, укажем папку его размещения. Прочие настройки можно оставить по умолчанию.

Щелчком правой кнопки мыши по корневой записи проекта в левой боковой панели вызовем контекстное меню. В нём выберем *Добавить существующие файлы ...*

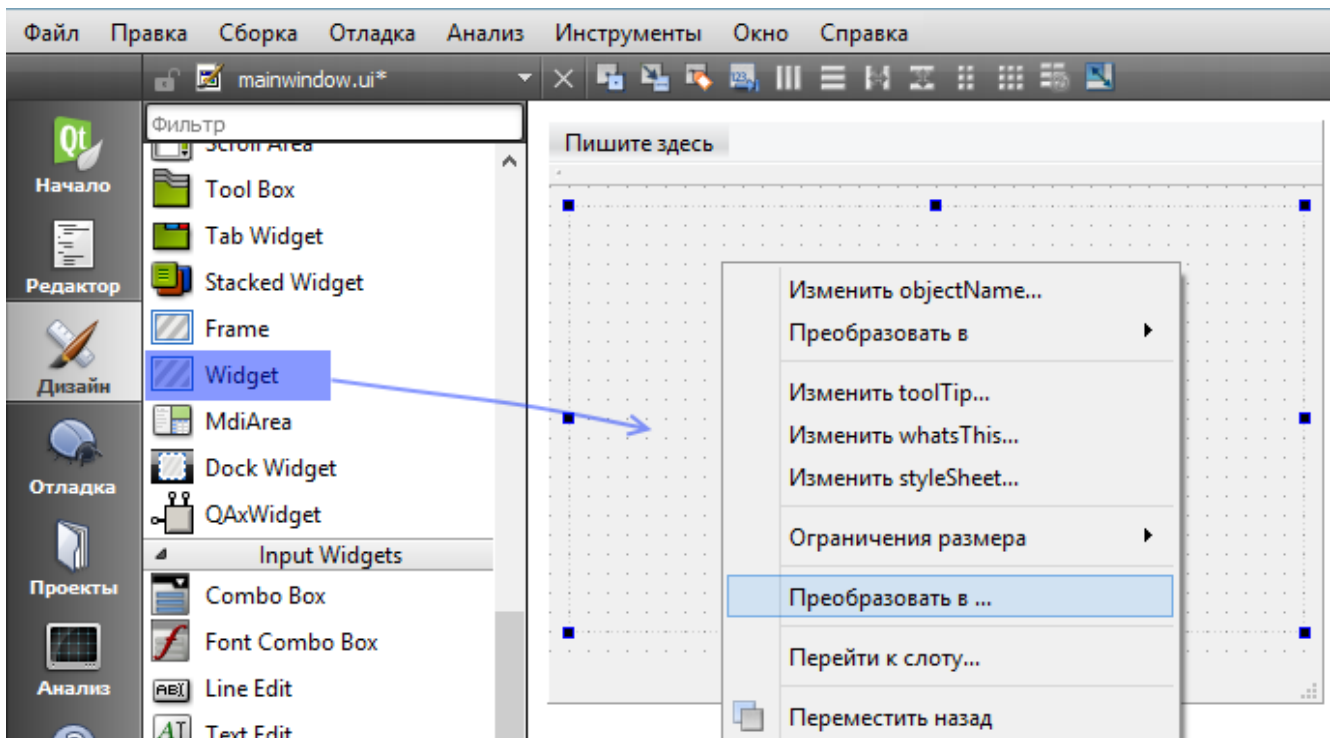


¹ Файлы данной библиотеки можно скачать с сайта разработчика <http://www.qcustomplot.com/>

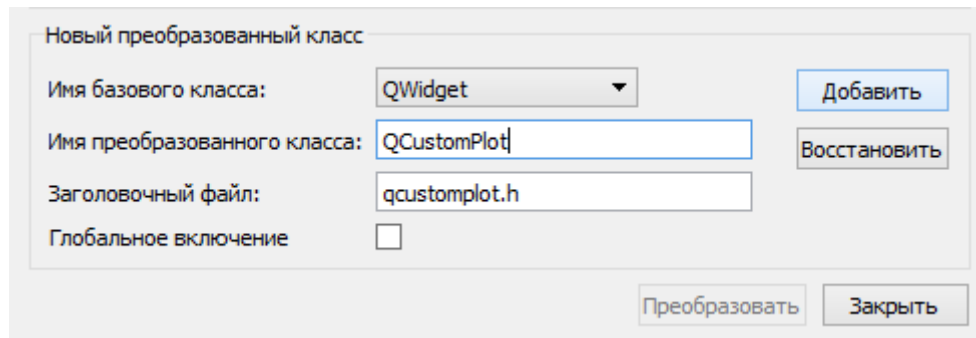
В появившемся диалоговом окне файла выберем файлы *qcustomplot.h* и *qcustomplot.cpp*. После чего файл *MyProject.pro* должен выглядеть как на рисунке снизу. Если используется библиотека Qt версии 5.0 и выше, нужно также добавить ключевое слово *printsupport*.



В режиме дизайнера разместим виджет *QWidget* на форме в нужном месте. Щелчком правой кнопки мыши по нему вызовем контекстное меню. В нём выберем *Преобразовать в ...*



В появившемся диалоговом окне введём *QCustomPlot* в поле *Имя преобразованного класса*. Нажмём кнопку *Добавить*, затем *Преобразовать*.



Никаких визуальных изменений виджета после этого последовать не должно. Однако во время запуска приложения на нём отобразится пустой прямоугольник с осями и линиями сетки.

Для демонстрации приведём листинг с кодом простого приложения, которое строит график функции $x \cdot \sin \frac{\pi x}{180}$ на промежутке от 0 до 360 и закрашивает на графике область под графиком на интервале [40, 320]. В данном приложении изменится только содержимое файла *mainwindow.cpp*, которое мы приводим. Пояснения к коду приведены в закомментированных строках.

Листинг III. Пример кода программы для построения графика заданной функции

```
/* mainwindow.cpp *****/
#include "graphwindow.h"
#include "ui_graphwindow.h"
#include "math.h"

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent), ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    // создаем 2 графика
    // график функции от 0 до 360
    ui->graph->addGraph();
    ui->graph->graph(0)->setPen(QPen(Qt::red,2));
    // график с закрашенной под ним областью на интервале [a,b]
    ui->graph->addGraph();
    ui->graph->graph(1)->setBrush(QBrush(QColor(0,0,255,30)));
    // будем разными способами добавлять данные на график
    // на первый (graph(0)) командой setData(),
    // передавая в неё массивы значений аргумента и функции
```

```

// на второй (graph(1)) командой addData(),
//      добавляя с помощью неё отдельную точку на график

double x_min = 0, x_max = 360;    // область построения графика
int N = 500;                      // число точек графика
double a = 40, b = 320;          // область закрашивания

// генерируем данные
QVector<double> x(N), y(N);
for (int i=0; i<N; ++i) {
    // значения аргумента в узлах сетки
    x[i] = i*(x_max-x_min)/N;
    // вычисляем значение функции
    y[i] = 10*sin(x[i]*M_PI/180)*(x[i]);
    // добавляем точку на график
    if (x[i] > a && x[i] < b)
        ui->graph->graph(1)->addData(x[i], y[i]);
}
// и добавляем данные:
ui->graph->graph(0)->setData(x, y);

// задаем имена осей координат
ui->graph->xAxis->setLabel("X");
ui->graph->yAxis->setLabel("Y");

// задаем размеры осей
ui->graph->xAxis->setRange(0, 360);
double y_min = * std::min_element(y.begin(), y.end());
double y_max = * std::max_element(y.begin(), y.end());
ui->graph->yAxis->setRange(y_min, y_max);

QVector<double> Ticks; // вектор с шагом в 30 градусов
int i = 0;
while(i<=360) { Ticks << i; i+=30; }

// выключаем автоматические отсчёты
ui->graph->xAxis->setAutoTicks(false);
// задаем в качестве новых отсчётов
//      значения элементов созданного вектора
ui->graph->xAxis->setTickVector(Ticks);
ui->graph->replot(); // отображение графика
}

MainWindow::~MainWindow() {
    delete ui; }

/*****

```

Скриншот работающего приложения приведён на рис. П1.

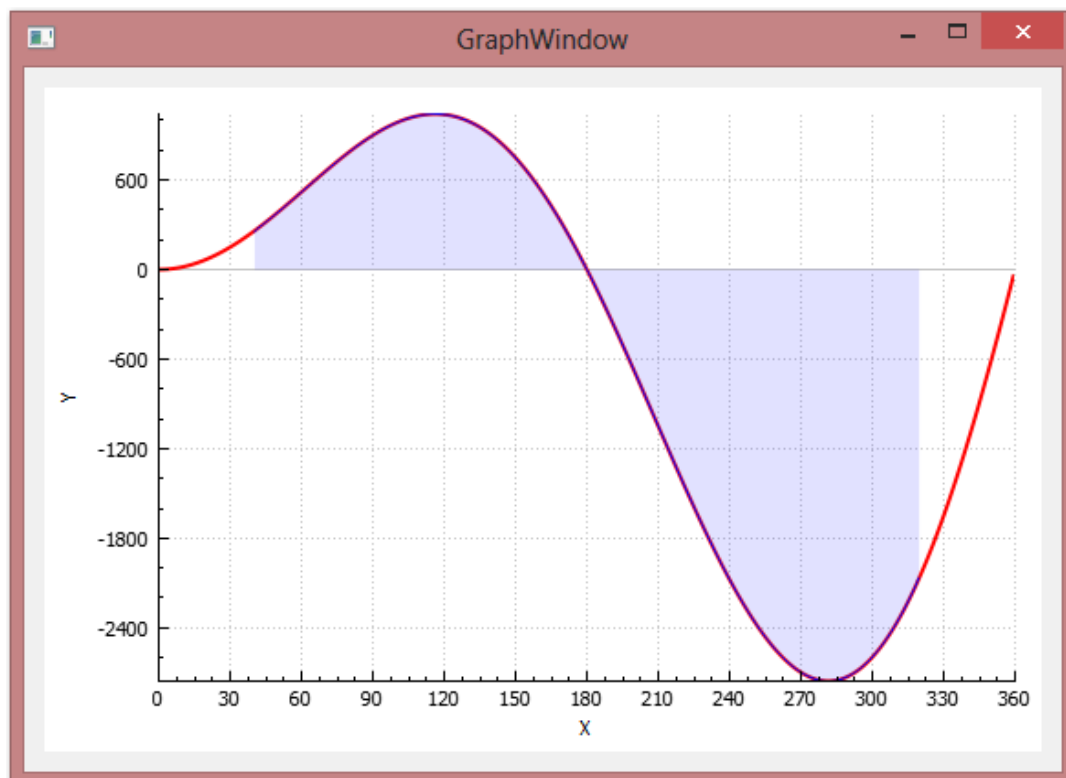


Рис. П1. Окно работающего приложения, строящего график функции $x \cdot \sin \frac{\pi x}{180}$ на промежутке от 0 до 360

II. Создание простой анимации в Qt Creator

Создадим простое приложение, в окне которого будет происходить анимация равномерного движения физического тела, представленного в виде круга. Рассмотрим подробно шаги создания такого приложения с использованием библиотеки Qt и редактора Qt Creator.

1. Создадим проект с названием *Movement*:

Файл → Новый файл или проект → Приложение → Приложение Qt Widgets.

Далее следуя инструкциям на экране введём название проекта, укажем папку его размещения. Прочие настройки можно оставить по умолчанию.

2. В обозревателе проекта выберем форму приложения (по умолчанию *mainwindow.ui*) и, кликнув дважды по ней, перейдём в режим дизайна. Здесь в панели виджетов в разделе *Display Widgets* необходимо выбрать *Graphics View* и пе-

ретащить его на форму. На нём и будет происходить отрисовка анимации. В блоке свойств виджета можно изменить его свойства, такие как положение на форме, размеры, название и прочие.

3. Добавим на форму элементы управления:

- Из раздела *Buttons* добавим элементы *Push Button* и *Check Box*.

В блоке свойств переименуем эти элементы в *btnGo* и *chTrajectory* соответственно (свойство *objectName*), установим значения «Запуск» и «Траектория» в поле свойства *text*.

Кнопка *btnGo* будет запускать анимацию, флажок *chTrajectory* будет отвечать за показ траектории движущегося тела, либо отображая, либо скрывая её.

- Из раздела *Input Widgets* добавим 2 элемента *Line Edit*.

Переименуем их в *edtV* и *edtA*.

В поле ввода *edtV* будет вводиться значение скорости. Значение по умолчанию установим для него равным 1 (свойство *text*).

В поле ввода *edtA* будет передаваться угол направления движения по отношению к горизонтальной оси. Установим значение угла равным 30.

Несмотря на то, что мы задали значения скорости и угла, пользователю будет предоставлена возможность поменять их на свои. Это сделано для удобства работы в программе, когда при первом запуске пользователь может запустить программу с параметрами по умолчанию.

- Из раздела *Display Widgets* добавим 2 элемента *Label*.

Переименуем их в *edtV* и *edtA* и расположим рядом с соответствующими полями ввода *edtV* и *edtA*. Значения свойства *text* для них определим как «Скорость» и «Угол» соответственно.

Должен получиться вид окна приложения, близкий к виду, приведённому на рис. П2.

4. Для прорисовки движущегося тела создадим класс *physBody*, унаследованный от класса *QGraphicsItem* библиотеки *Qt*, предназначенного для работы с графическими объектами. В этом классе определим функции определения области

рисования *boundingRect*, непосредственно процедуру рисования *paint*, объявим поле *radius* в разделе *public* для возможности задавать размер тела.

Добавление класса производится командами Файл → Новый файл или проект → C++ → C++ Class. При этом создадутся файл исходного кода C++ и заголовочный файл. Следует сразу указать имя базового класса *QGraphicsItem* в поле *Base Class* диалогового окна. Содержание исполняемого и заголовочного файлов для класса *physBody* приведено в листингах П2 – П3.

Листинг П2. Содержание заголовочного файла для класса *physBody*

```
/* physbody.h *****/
#ifndef PHYSBODY_H
#define PHYSBODY_H

#include <QPainter>
#include <QGraphicsScene>
#include <QGraphicsItem>

class physBody : public QGraphicsItem
{
public:
    physBody();        // конструктор
    ~physBody();       // деструктор
    QRectF boundingRect() const;
    void paint(QPainter *painter,
               const QStyleOptionGraphicsItem *option,
               QWidget *widget);
    double radius;     // размер тела
};

#endif // PHYSBODY_H
/*****
```

Листинг П3. Содержание исполняемого файла для класса *physBody*

```
/* physbody.cpp *****/
#include "physbody.h"

physBody::physBody()
{
}
```

```

// Определение границ объекта "Планета" на графической сцене
QRectF physBody::boundingRect() const
{
    return QRect(0,0,2*radius,2*radius);
}

// процедура отрисовки планеты
void physBody::paint(QPainter *painter,
                    const QStyleOptionGraphicsItem *option,
                    QWidget *widget)
{
    QPen pen(Qt::black, 1); // цвет контура тела (чёрный)
    QBrush brush(Qt::blue); // цвет тела (синий)
    painter->setPen(pen);
    painter->setBrush(brush);
    // прорисовка тела в виде окружности
    painter->drawEllipse(boundingRect());
}

physBody::~physBody()
{
}

/*****

```

5. Запуск анимации должен производиться при нажатии на кнопку «*Запустить*». Чтобы создать процедуру, которая будет выполняться при нажатии на эту кнопку, необходимо в режиме дизайна кликнуть на ней правой кнопкой мыши, из раскрывающегося меню нажать *Перейти к слоту...*, затем в открывшемся окне выбрать событие *clicked()* и нажать ОК. После этого *Qt Creator* автоматически перейдёт в режим редактирования файла *mainwindow.cpp*, где будет создана пустая процедура *MainWindow::on_btnGo_clicked()*. Реализация данной процедуры описана в листинге П4.

6. Для создания анимации нам необходимо будет использовать библиотеки: *QGraphicsScene* – для работы с графическими объектами и *QTimer* – для работы с объектом таймера, позволяющим задать процедуру, выполнение которой производится через заданные интервалы времени. Объявление объекта графической сцены *scene* и таймера *timer* поместим в заголовочный файл класса *MainWindow*

окна приложения. Также объявим здесь процедуру *showEvent*, которую переопределим затем в исполняемом файле. Она выполняется при отображении окна приложения, поэтому в ней будет прописано определение первоначального положения системы.

В начале заголовочного файла *mainwindow.h* подключим вышеуказанные библиотеки. Следом поместим объявление структуры *vector* для работы с проекциями физических величин. А также в объявлении класса *mainwindow* изменим блоки, *protected*, *private* и *private slots*, объявив процедуру *move*, указатели на объект графической сцены и на объект таймера. Содержимое полученного в результате заголовочного файла приведено в листинге П4.

Процедура *move* отвечает за определения новых положений планет в текущем кадре анимации, поэтому она связана с сигналом таймера *timeout()* и выполняется на каждый его отсчёт.

7. Реализации процедур *move*, *showEvent* и *on_btnGo_clicked* необходимо прописать в файле исходного кода *mainwindow.cpp*. В листинге П5 приведено его содержимое.

Листинг П4. Содержание заголовочного файла главного окна приложения *Movement*

```
/* mainwindow.h *****/
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QGraphicsScene>
#include <QTimer>

struct vector {
    // горизонтальная составляющая
    double x;
    // вертикальная составляющая
    double y;
};

namespace Ui{
class MainWindow;
}
```

```

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = 0);
    ~MainWindow();

protected:
    void showEvent(QShowEvent * event);

private slots:
    void on_btnGo_clicked();
    void move();

private:
    Ui::MainWindow *ui;
    QGraphicsScene *scene;
    QTimer *timer;

};

#endif // MAINWINDOW_H
/*****

```

Листинг П5. Содержание исполняемого файла главного окна приложения *Movement*

```

/* mainwindow.cpp *****/

#include "mainwindow.h"
#include "ui_mainwindow.h"

#include "physbody.h"
#include <math.h> // для работы с математическими функциями

bool IsRun = false; // флаг запуска анимации
double rad = 10;    // радиус шарика
double dt = 10;     // временной шаг
double v0;          // начальная скорость
double angle;       // начальное направление скорости
vector v;           // вектор скорости
vector r;           // радиус-вектор
physBody * ball = new physBody(); // физическое тело
QImage * img;       // картинка для фона
QImage bgr, bgr_clear; // фон с траекторией и без неё
QPainterPath traj;  // траектория
QPainter * p;       // объект рисовальщик

```

```

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    //создадим сцену
    scene = new QGraphicsScene(ui->graphicsView);

    // anti-aliasing
    ui->graphicsView->setRenderHint(QPainter::Antialiasing);

    // создаём картинку с фоном, будем использовать её как текстуру
    int imgSize = 400;    // размер картинки
    img = new QImage(imgSize, imgSize, QImage::Format_RGB16);
    QPainter painter(img);           // рисовальщик
    painter.setPen(Qt::NoPen);        // будем рисовать без контура
    painter.setBrush(QColor(255, 255, 224)); // цвет фона
    painter.drawRect(0, 0, imgSize, imgSize);

    //создадим таймер
    timer = new QTimer(this);
    connect(timer, SIGNAL(timeout()), this, SLOT(move()));
}

void MainWindow::move()
{
    // изменение направления скорости при отскоке
    if (r.x+rad > scene->width()-1 || r.x-rad < 0) {
        v.x = -v.x; }
    if (r.y+rad > scene->height()-1 || r.y-rad < 0) {
        v.y = -v.y; }

    // определение новых координат тела
    r.x += v.x*dt;
    r.y += v.y*dt;
    ball->setPos(r.x-rad, scene->height()-(r.y+rad));
    // добавление участка траектории
    traj.lineTo(r.x, scene->height()-r.y);
    p->drawPath(traj);

    if (ui->chTrajectory->isChecked()) {
        scene->setBackgroundBrush(bgr); }
    else {
        scene->setBackgroundBrush(bgr_clear);}
}

```

```

void MainWindow::showEvent(QShowEvent *)
{
    // начальное положение тела
    r.x = rad;    r.y = rad;

    // задаём область графической сцены
    scene->setSceneRect(0,0,ui->graphicsView->height()-2,
                        ui->graphicsView->height()-2);
    ui->graphicsView->setScene(scene);

    // устанавливаем в качестве фона изображение img
    bgr = img->scaledToHeight(ui->graphicsView->height());
    bgr_clear = img->scaledToHeight(ui->graphicsView->height());
    scene->setBackgroundBrush(bgr);

    // создание и настройка рисовальщика траектории
    p = new QPainter(&bgr);
    QPen pen(Qt::black,2);
    p->setPen(pen);
    traj.moveTo(r.x, scene->height()-r.y);

    // добавление физического тела
    ball->radius = rad;
    ball->setPos(r.x-rad, scene->height()-(r.y+rad));
    scene->addItem(ball);
}

void MainWindow::on_btnGo_clicked()
{
    // модуль начальной скорости
    v0 = ui->edtV->text().toDouble();
    // угол наклона начальной скорости к горизонтальной оси
    angle = ui->edtA->text().toDouble()/180*M_PI;
    // определение проекций начальной скорости
    v.x = v0*cos(angle);    v.y = v0*sin(angle);

    IsRun = !IsRun;
    if (IsRun) {
        timer->start(dt);
        ui->btnGo->setText("Стоп");
    }
    else {
        timer->stop();
        ui->btnGo->setText("Запуск");
    }
}

```

```
MainWindow::~MainWindow()
{
    delete ui;
}
/*****
```

Скриншот работающего приложения приведён на рис. П2.

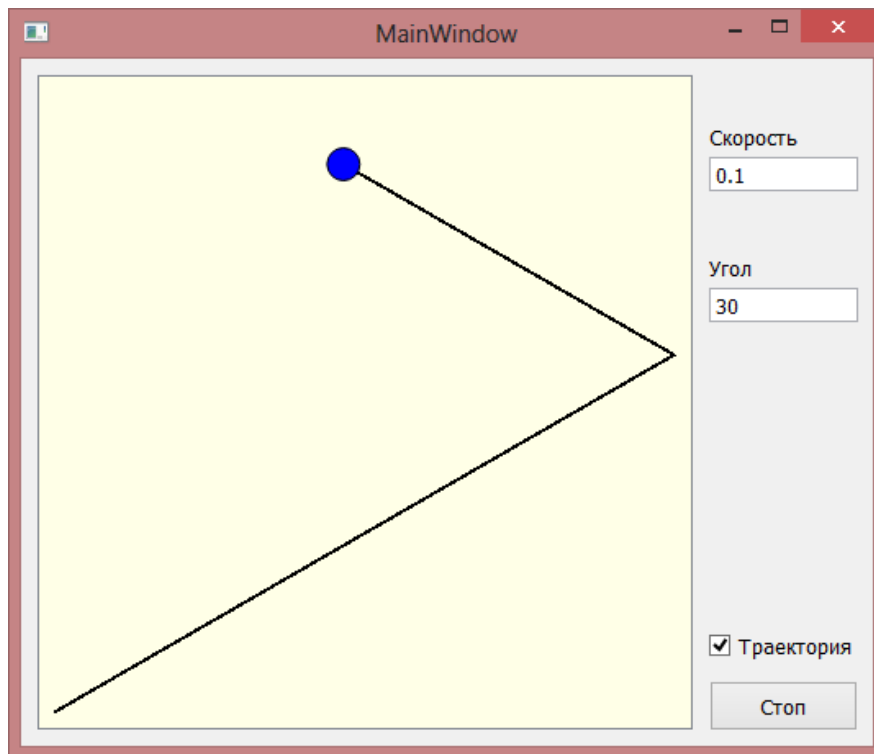


Рис. П2. Окно работающего приложения *Movement*

III. Создание простейшей модели Солнечной системы в Qt Creator

Создадим² простейшую модель солнечной системы, в которой движение планет будет осуществляться по окружностям с заданным периодом обращения. Рассмотрим подробно шаги создания компьютерной программы, анимирующую движение объектов такой модели с использованием *Qt Creator* и библиотеки *Qt*.

1. Создадим проект с названием *Planets*:

Файл → Новый файл или проект → Приложение → Приложение Qt Widgets.

² Частично использованы материалы форума: <http://programmersforum.ru/showthread.php?t=25052>

Далее, следуя инструкциям на экране, зададим основные параметры проекта.

2. В обозревателе проекта выберем форму приложения, перейдём в режим дизайна. В панели виджетов выберем элемент *Graphics View* и добавим на форму. Переименуем его в *CosmosView*. На нём и будет происходить отрисовка солнечной системы.

3. Для определения параметров планет удобно создать структуру. Назовём её *planet*, а её объявление поместим в заголовочный файл *planet.h*. Также в нём зададим массивы с параметрами планет. Пусть значение элемента с индексом 0 в каждом из этих массивов будет соответствовать Солнцу. Остальные индексы будут соответствовать номерам планет в порядке удаления от Солнца. Содержимое файла *planet.h* приведено в листинге П6.

Добавление заголовочного файла производится командами Файл → Новый файл или проект → C++ → C++ Header File.

4. Для прорисовки планет воспользуемся созданным ранее классом *physBody*. Добавим в его заголовочном файле объявление поля *color* в разделе *public* для возможности задавать цвет планеты в коде основной программы:

```
QColor color;
```

В исполняемом файле заменим строку объявления цвета кисти на следующую:

```
QBrush brush(color);
```

Листинг П6. Содержание заголовочного файла *planet.h*

```
/* planet.h *****/
#ifndef PLANET
#define PLANET

// радиусы планет в пикселях (выбираем пропорционально реальным)
int radiuses[10] = {15, 3, 5, 6, 4, 12, 10, 8, 7, 2};

// названия планет
const char * names[10] =
    {"Солнце", "Меркурий", "Венера", "Земля", "Марс",
     "Юпитер", "Сатурн", "Нептун", "Уран", "Плутон"};

// радиусы орбит в пикселях (выбираем пропорционально реальным)
int orbits[10] = {0, 25, 40, 55, 70, 100, 145, 190, 240, 290};
```



```

// периоды обращения планет в сутках (реальный параметр)
double periods[10] = {0, 88, 225, 365, 687,
                      4332, 10762, 30689, 60201, 90485};

struct planet {
    double x;          // текущие координаты
    double y;          // для удобства
    double period;     // период обращения вокруг солнца (в сутках)
    double orbit;      // радиус орбиты
    double angle;      // текущее положение планеты на орбите
    double radius;     // радиус планеты
    const char * name; // имя планеты
};

#endif // PLANET
/*****

```

5. Для создания анимации нам необходимо будет использовать библиотеки: *QGraphicsScene* – для работы с графическими объектами и *QTimer* – для работы с объектом таймера, позволяющим задать процедуру, выполнение которой производится через заданные интервалы времени. Объявление объекта графической сцены *scene* и таймера *timer* поместим в заголовочный файл класса *MainWindow* окна приложения. Также в блоке *protected* объявим процедуру *showEvent*, которую переопределим затем в исполняемом файле. Для примера см. в листинг П4 с кодом заголовочного файла проекта *Movement*. Однако в данном проекте в заголовочном файле *mainwindow.h* объявим в блоке *private slots* также процедуры *move* и *setPlanetsPos*:

```

private slots:
    void move();
    void setPlanetsPos();

```

Процедура *move* отвечает за определения новых положений планет в текущем кадре анимации, поэтому она связана с сигналом таймера *timeout()* и выполняется на каждый его отсчёт. Процедура *setPlanetsPos* устанавливает положения планет на текущем шаге. Содержание исполняемого файла *mainwindow.cpp* проекта *Planets* приведено в листинге П7.

```

/* mainwindow.cpp *****/
#include "mainwindow.h"
#include "ui_mainwindow.h"

#include "physbody.h"
#include "planet.h"
#include <math.h> // для работы с математическими функциями

QImage * cosmos;          // картинка для фона
int dt = 20;              // шаг времени анимации
physBody *p_[10];         // массив графических объектов
planet p[10];             // массив параметров планет
QColor colors[10] = {     // массив значений цвета планет
    Qt::yellow,           // Солнце
    Qt::darkYellow,       // Меркурий
    Qt::cyan,             // Венера
    Qt::green,            // Земля
    Qt::red,              // Марс
    Qt::darkCyan,         // Юпитер
    Qt::magenta,          // Сатурн
    Qt::blue,             // Уран
    Qt::gray,             // Нептун
    Qt::cyan };           // Плутон

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    // создаём графическую сцену
    scene = new QGraphicsScene(ui->CosmosView);

    // anti-aliasing
    ui->CosmosView->setRenderHint(QPainter::Antialiasing);

    // создаём картинку для использования её как текстуры фона
    int imgSize = 400;     // размер картинки
    int starsCnt = 200;    // количество "звёзд"
    cosmos = new QImage(imgSize, imgSize, QImage::Format_RGB16);
    QPainter painter(cosmos); // рисовальщик
    painter.setPen(Qt::NoPen); // будем рисовать без контура
    painter.setBrush(Qt::black); // цвет космоса

```

```

// чёрный квадрат для космоса
painter.drawRect(0,0,imgSize,imgSize);

// добавим случайным образом звёзды в виде белых пикселей
painter.setPen(Qt::white);
for (int i=0; i<starsCnt; i++) {
    painter.drawPoint(rand()%imgSize+1,rand()%imgSize+1);
}

// создаём таймер
timer = new QTimer(this);
connect(timer, SIGNAL(timeout()), this, SLOT(move()));

// задаём параметры планет
for (int i=0; i<10; i++) {
    p_[i] = new physBody();
    p_[i]->radius = radiuses[i];
    p_[i]->color = colors[i];
    p[i].radius = radiuses[i];
    p[i].name = names[i];
    p[i].orbit = orbits[i];
    p[i].period = periods[i];
    p[i].angle = 0;
}
}

void MainWindow::move()
{
    // задаём значение временного шага deltaT движения планет
    // через коэффициент "масштаба" времени timeScale, который равен
    // количеству суток в одной секунде реальной системы отсчёта
    double timeScale = 100;
    double deltaT = timeScale * 1.e-3*dt;

    // определяем новые положения планет на орбите
    for(int i=1; i<10; i++) {
        p[i].angle += 2*M_PI * deltaT/p[i].period;
        if(p[i].angle > 2*M_PI) p[i].angle -= 2*M_PI;
    }
    // задаём новые положения планет
    setPlanetsPos();
}

void MainWindow::showEvent(QShowEvent *)
{
    // задаём область графической сцены

```

```

scene->setSceneRect(0,0, ui->CosmosView->height()-2,
                    ui->CosmosView->height()-2);

ui->CosmosView->setScene(scene);

// устанавливаем в качестве фона изображение cosmos
QImage bgr = cosmos->scaledToHeight(ui->CosmosView->height());
scene->setBackgroundBrush(bgr);

// задаём положение планет
setPlanetsPos();

// добавляем все объекты планет и солнце на графическую сцену
for (int i=0; i<10; i++) {
    scene->addItem(p_[i]);
}
// запускаем таймер
timer->start(dt);
}

void MainWindow::setPlanetsPos()
{
    // помещаем Солнце в центр
    p[0].x = scene->width()/2;
    p[0].y = scene->height()/2;

    // задаём положение планет относительно Солнца
    p_[0]->setPos(p[0].x-p[0].radius,
                 p[0].y-p[0].radius);
    for (int i=1; i<10; i++) {
        p[i].x = p[0].x + p[i].orbit*cos(p[i].angle);
        p[i].y = p[0].y + p[i].orbit*sin(p[i].angle);
        p_[i]->setPos(p[i].x-p[i].radius,
                     p[i].y-p[i].radius);
    }
}

MainWindow::~MainWindow()
{
    delete ui;
}

/*****

```

Скриншот работающего приложения приведён на рис. ПЗ.

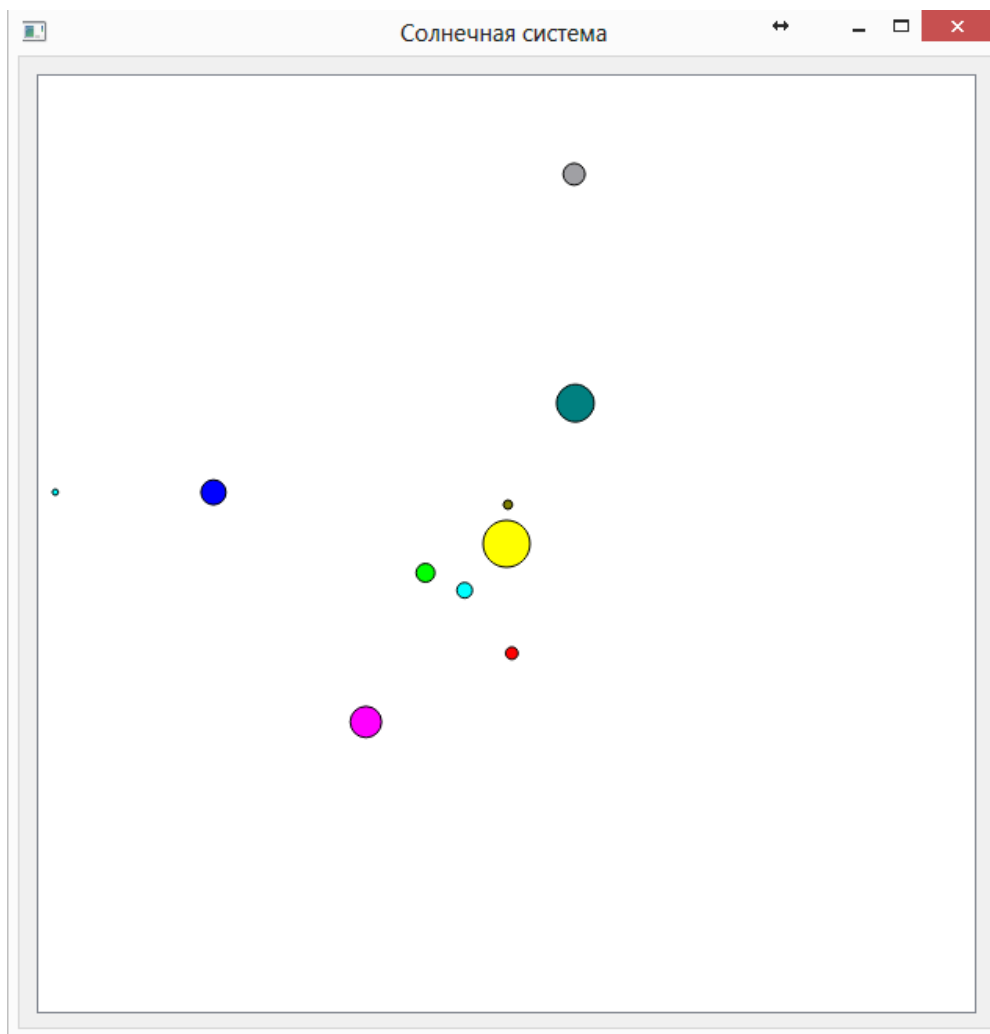


Рис. П3. Окно работающего приложения *Planets*
(фон изменён на полотно белого цвета)

Заключение

В данном пособии рассматриваются возможности использования вычислительной техники применительно к процессу изучения физики, в частности в компьютерной обработке результатов физического эксперимента, а также при компьютерном моделировании физических явлений. В результате освоения материала данного пособия студент должен получить практические навыки численной обработки прямых измерений на компьютере, навыки численного решения дифференциальных уравнений, создания физических моделей и их визуализации.

Библиографический список

1. *Бланишет, Ж.* Qt 4: программирование GUI на C++ : пер. с англ. Ж. Бланшет, М. Саммерфилд. 2-е изд., доп. – М. : КУДИЦ-ПРЕСС, 2008. – 736 с.
2. *Волков, В. Б.* Понятный самоучитель Excel 2010 / В. Б. Волков. – СПб. : Питер, 2010. – 256 с.
3. *Горлач, В. В.* Обработка, представление, интерпретация результатов измерений : учеб. пособие / В. В. Горлач, В. Л. Егоров, Н. А. Иванов / под ред. В. В. Горлача. – Омск : Изд-во СибАДИ, 2006. – 83 с.
4. *Гулд, Х.* Компьютерное моделирование в физике : в 2 ч. / Х. Гулд, Я. Тобочник. – М. : Мир, 1990. – Ч. 1. – 350 с.; Ч. 2. – 400 с.
5. *Майер, Р. В.* Компьютерное моделирование физических явлений / Р. В. Майер. – Глазов : ГГПИ, 2009. – 112 с.
6. NASA. [Офиц. сайт]. – Режим доступа : <http://nssdc.gsfc.nasa.gov/planetary/factsheet/> (дата обращения: 29.05.2015).
7. *Планетные системы.* [Электронный ресурс]. – Режим доступа : http://www.allplanets.ru/solar_sistem.htm (дата обращения: 29.05.2015).
8. *Третьяк, Л. Н.* Обработка результатов наблюдений : учеб. пособие / Л. Н. Третьяк – Оренбург: ГОУ ОГУ, 2004. – 171 с.
9. *Шлее, М.* Qt 5.3. Профессиональное программирование на C++ / М. Шлее – СПб. : БХВ-Петербург, 2015. – 928 с.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1. КОМПЬЮТЕРНАЯ ОБРАБОТКА ЭКСПЕРИМЕНТАЛЬНЫХ ДАННЫХ.....	4
§ 1. Погрешности измерений	4
Введение	4
Расчет случайной погрешности в прямых измерениях	6
§ 2. Графическое представление результатов измерений.....	7
Основные правила оформления графиков	7
Метод наименьших квадратов	9
Расчет случайной погрешности средствами Excel	11
Расчет погрешности при косвенных измерениях.....	15
Задания для самостоятельного выполнения	16
2. КОМПЬЮТЕРНОЕ МОДЕЛИРОВАНИЕ ФИЗИЧЕСКИХ ЯВЛЕНИЙ	18
§ 3. Вычисление определенных интегралов	18
Методы численного интегрирования	18
Решение задач на распределение Максвелла.	22
Задания для самостоятельного выполнения.....	23
§ 4. Математическое моделирование движения и расчёт траектории	24
Введение	24
Метод Эйлера.....	25
Движение физических тел в поле сил	26
Моделирование движения системы небесных тел.....	34
Задания для самостоятельного выполнения.....	37
§ 5. Математическое моделирование колебательных движений	39
Линейные колебания	40
Математический маятник	42
Задания для самостоятельного выполнения.....	43
§ 6. Моделирование векторных полей.....	44
Силовые линии	45
Силовые линии электрического поля	46
Силовые линии магнитного поля	49
Задания для самостоятельного выполнения.....	52
§ 7. Построение изолиний векторного поля	55
Изолинии потенциала электростатического поля	55
Задания для самостоятельного выполнения	57
ПРИЛОЖЕНИЯ	59
I. Работа с графиками в Qt с использованием библиотеки QCustomPlot	59
II. Создание простой анимации в Qt Creator	63
III. Создание простейшей модели Солнечной системы в Qt Creator	71
Заключение	77
Библиографический список	78

Учебное издание

Алёшин Максим Сергеевич

Насыров Вячеслав Вячеславович

ВЫЧИСЛИТЕЛЬНАЯ ФИЗИКА

Учебное пособие

Отпечатано с авторского оригинала-макета.

Дизайнер обложки

Подписано в печать ..06.16. Формат 60х84 $\frac{1}{16}$. Бумага писчая. Гарнитура «Таймс».

Печать цифровая. Усл. печ. л. 5. Тираж 100 экз. Заказ

Издательство Тихоокеанского государственного университета.

680035, Хабаровск, ул. Тихоокеанская, 136.

Отдел оперативной полиграфии издательства Тихоокеанского государственного университета.

680035, Хабаровск, ул. Тихоокеанская, 136.